

परीक्षा प्लस

COMPUTER

DATA STRUCTURE



Android App On
GOOGLE PLAY



/PARIKSHAPLUS



LIVE.PARIKSHAPLUS.ORG



/PARIKSHAPLUS



9450006007 | 9129697080

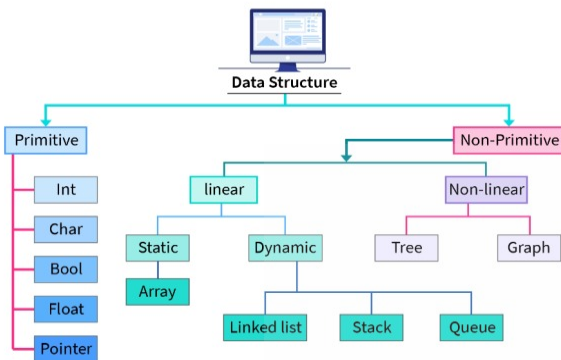


PARIKSHAPLUS.ORG

What is Data Structure:

A data structure is a storage that is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.

Classification of Data Structure: The following figure shows the different classifications of Data Structures.



Primitive Data Structures

- Primitive Data Structures** are the data structures consisting of the numbers and the characters that come **in-built** into programs.
- These data structures can be manipulated or operated directly by machine-level instructions.
- Basic data types like **Integer**, **Float**, **Character**, and **Boolean** come under the Primitive Data Structures.
- These data types are also called **Simple data types**, as they contain characters that can't be divided further

Non-Primitive Data Structures

- Non-Primitive Data Structures** are those data structures derived from Primitive Data Structures.
- These data structures can't be manipulated or operated directly by machine-level instructions.
- The focus of these data structures is on forming a set of data elements that is either **homogeneous** (same data type) or **heterogeneous** (different data types).
- Based on the structure and arrangement of data, we can divide these data structures into two sub-categories -

a. Linear Data Structures

b. Non-Linear Data Structures

Linear data structure: Data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure. *Examples of linear data structures are array, stack, queue, linked list, etc.*

- Static data structure:** Static data structure has a fixed memory size. It is easier to access the elements in a static data structure. *An example of this data structure is an array.*
- Dynamic data structure:** In the dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code. *Examples of this data structure are queue, stack, etc.*
- Non-linear data structure:** Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. In a non-linear data structure, we can't traverse all the elements in a single run only. *Examples of non-linear data structures are trees and graphs.*

Introduction to Linear Data Structures

Linear Data Structures are a type of data structure in computer science where data elements are arranged sequentially or linearly. Each element has a previous and next adjacent, except for the first and last elements.

Characteristics of Linear Data Structure:

- Sequential Organization:** In linear data structures, data elements are arranged sequentially, one after the other. Each element has a unique predecessor (except for the first element) and a unique successor (except for the last element)
- Order Preservation:** The order in which elements are added to the data structure is preserved. This means that the first element added will be the first one to be accessed or removed, and the last element



added will be the last one to be accessed or removed.

- **Fixed or Dynamic Size:** Linear data structures can have either fixed or dynamic sizes. Arrays typically have a fixed size when they are created, while other structures like linked lists, stacks, and queues can dynamically grow or shrink as elements are added or removed.
- **Efficient Access:** Accessing elements within a linear data structure is typically efficient. For example, arrays offer constant-time access to elements using their index.

Linear data structures are commonly used for organising and manipulating data in a sequential fashion. Some of the most common linear data structures include:

- **Arrays:** A collection of elements stored in contiguous memory locations.
- **Linked Lists:** A collection of nodes, each containing an element and a reference to the next node.
- **Stacks:** A collection of elements with **Last-In-First-Out (LIFO)** order.
- **Queues:** A collection of elements with **First-In-First-Out (FIFO)** order.

1. Array

An array is a collection of items of same data type stored at contiguous memory locations.

Memory Location									
200	201	202	203	204	205	206	■	■	■
U	B	F	D	A	E	C	■	■	■
0	1	2	3	4	5	6	■	■	■
Index									

Array

Characteristics of Array Data Structure:

- **Homogeneous Elements:** All elements within an array must be of the same data type.
- **Contiguous Memory Allocation:** In most programming languages, elements in an array are stored in contiguous (adjacent) memory locations.
- **Zero-Based Indexing:** In many programming languages, arrays use zero-based indexing, which means that the first element is accessed with an index of 0, the second with an index of 1, and so on.
- **Random Access:** Arrays provide constant-time ($O(1)$) access to elements. This means that regardless of the size of the array, it takes the same amount of time to access any element based on its index.

Types of arrays:

- **One-Dimensional Array:** This is the simplest form of an array, which consists of a single row of elements, all of the same data type. Elements in a 1D array are accessed using a single index.

Index	0	1	2	3	4	5
Element	5	10	20	25	30	35

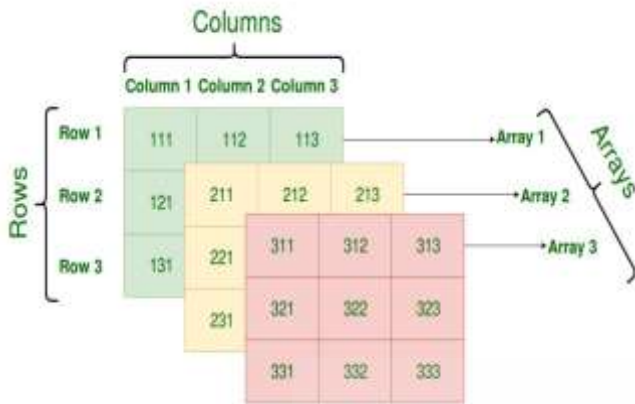
One-Dimensional Array

- **Two-Dimensional Array:** A two-dimensional array, often referred to as a matrix or 2D array, is an array of arrays. It consists of rows and columns, forming a grid-like structure. Elements in a 2D array are accessed using two indices, one for the row and one for the column.

	Col		
	0	1	2
Row 0	5	10	20
Row 1	25	30	35
Row 2	1	3	4

Two-Dimensional Array:

- **Multi-Dimensional Array:** Arrays can have more than two dimensions, leading to multi-dimensional arrays. These are used when data needs to be organized in a multi-dimensional grid.



Multi-Dimensional Array

Types of Array operations:

- **Accessing Elements:** Accessing a specific element in an array by its index is a constant-time operation. It has a time complexity of $O(1)$.
- **Insertion:** Appending an element to the end of an array is usually a constant-time operation, $O(1)$ but insertion at the beginning or any specific index takes $O(n)$ time because it requires shifting all of the elements.
- **Deletion:** Same as insertion, deleting the last element is a constant-time operation, $O(1)$ but deletion of element at the beginning or any specific index takes $O(n)$ time because it requires shifting all of the elements.
- **Searching:** Linear Search takes $O(n)$ time which is useful for unsorted data and Binary Search takes $O(\log n)$ time which is useful for sorted data.

Linked List

A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers. In simple words, a linked list consists of nodes where each node contains a data field and a reference(link) to the next node in the list.

Types Of Linked List:

1. Singly Linked List

It is the simplest type of linked list in which every node contains some data and a pointer to the next node of the same data type.

The node contains a pointer to the next node means that the node stores the address of the next node in the sequence. A single linked list allows the traversal of data only in one way. Below is the image for the same:

Singly Linked List



2. Doubly Linked List

A doubly linked list or a two-way linked list is a more complex type of linked list that contains a pointer to the next as well as the previous node in sequence.

Therefore, it contains three parts of data, a pointer to the next node, and a pointer to the previous node. This would enable us to traverse the list in the backward direction as well. Below is the image for the same:

Doubly Linked List



Time Complexity:

The time complexity of the push() function is $O(1)$ as it performs constant-time operations to insert a new node at the beginning of the doubly linked list. The time complexity of the printList() function is $O(n)$ where n is the number of nodes in the doubly linked list. This is because it traverses the entire list twice, once in the forward direction and once in the backward direction. Therefore, the overall time complexity of the program is $O(n)$.

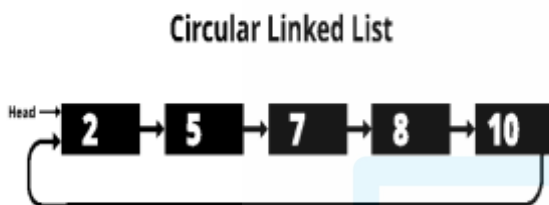
Space Complexity:

The space complexity of the program is $O(n)$ as it uses a doubly linked list to store the data, which requires n nodes. Additionally, it uses a constant amount of auxiliary space to create a new node in the push() function. Therefore, the overall space complexity of the program is $O(n)$.

3. Circular Linked List

A circular linked list is that in which the last node contains the pointer to the first node of the list.

While traversing a circular linked list, we can begin at any node and traverse the list in any direction forward and backward until we reach the same node we started. Thus, a circular linked list has no beginning and no end. Below is the image for the same:

**Time Complexity:**

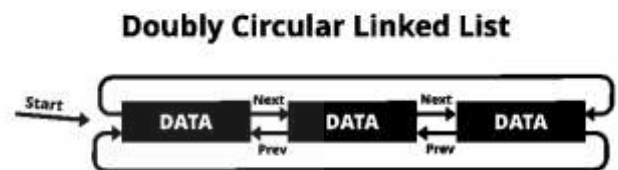
Insertion at the beginning of the circular linked list takes $O(1)$ time complexity. Traversing and printing all nodes in the circular linked list takes $O(n)$ time complexity where n is the number of nodes in the linked list. Therefore, the overall time complexity of the program is $O(n)$.

Auxiliary Space:

The space required by the program depends on the number of nodes in the circular linked list. In the worst-case scenario, when there are n nodes, the space complexity of the program will be $O(n)$ as n new nodes will be created to store the data. Additionally, some extra space is required for the temporary variables and the function calls. Therefore, the auxiliary space complexity of the program is $O(n)$.

4. Doubly Circular linked list

A Doubly Circular linked list or a circular two-way linked list is a more complex type of linked list that contains a pointer to the next as well as the previous node in the sequence. The difference between the doubly linked and circular doubly list is the same as that between a singly linked list and a circular linked list. The circular doubly linked list does not contain null in the previous field of the first node. Below is the image for the same:



Below is the structure of the Doubly Circular Linked List:

Time Complexity:

Insertion at the beginning of a doubly circular linked list takes $O(1)$ time complexity. Traversing the entire doubly circular linked list takes $O(n)$ time complexity, where n is the number of nodes in the linked list. Therefore, the overall time complexity of the program is $O(n)$.

Auxiliary space:

The program uses a constant amount of auxiliary space, i.e., $O(1)$, to create and traverse the doubly circular linked list. The space required to store the linked list grows linearly with the number of nodes in the linked list. Therefore, the overall auxiliary space complexity of the program is $O(1)$.

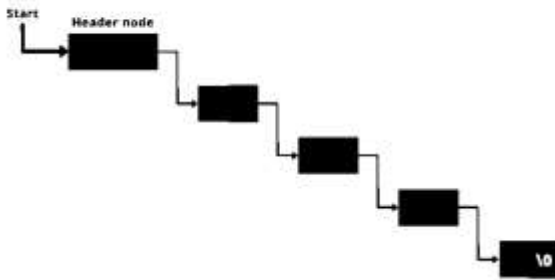
5. Header Linked List:

A header linked list is a special type of linked list that contains a header node at the beginning of the list.

So, in a header linked list **START** will not point to the first node of the list but **START** will contain the address of the header node. Below is the image for Grounded Header Linked List:



Grounded Headed Linked List



Time Complexity:

The time complexity of creating a new node and inserting it at the end of the linked list is $O(1)$. The time complexity of traversing the linked list to display its contents is $O(n)$, where n is the number of nodes in the list. Therefore, the overall time complexity of creating and traversing a header linked list is $O(n)$.

Auxiliary Space:

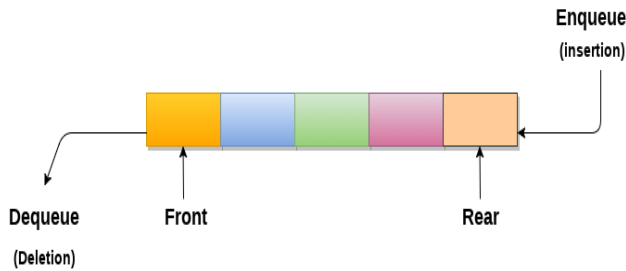
The space complexity of the program is $O(n)$, where n is the number of nodes in the linked list. This is because we are creating n nodes, each with a fixed amount of space required for storing the node information and a pointer to the next node. Therefore, the overall auxiliary space complexity of the program is $O(n)$.

Operation	One-Dimensional Array	Two-Dimensional Array
Accessing an Element by Index	Time: $O(1)$	Time: $O(1)$
	Space: $O(1)$	Space: $O(1)$
Inserting an Element at the End	Time: $O(1)$ (Amortized)	Time: $O(1)$
	Space: $O(1)$	Space: $O(1)$
Inserting an Element at the Beginning	Time: $O(n)$	Time: $O(1)$
	Space: $O(n)$	Space: $O(n)$
Searching for an Element (Linear Search)	Time: $O(n)$	Time: $O(m * n)$

	Space: $O(1)$	Space: $O(1)$
Transposing a Matrix	N/A	Time: $O(m * n)$
	N/A	Space: $O(m * n)$
Deletion of an Element	Time: $O(n)$	Time: $O(m * n)$
	Space: $O(1)$	Space: $O(1)$

Operation	Time Complexity (Singly Linked List)	Time Complexity (Doubly Linked List)	Space Complexity
Accessing by Index	$O(n)$	$O(n)$	$O(1)$
Insertion at Beginning	$O(1)$	$O(1)$	$O(1)$
Insertion at End	$O(n)$	$O(1)$	$O(1)$
Insertion at Given Position	$O(n)$	$O(n)$	$O(1)$
Deletion at Beginning	$O(1)$	$O(1)$	$O(1)$
Deletion at End	$O(n)$	$O(1)$	$O(1)$
Deletion at Given Position	$O(n)$	$O(n)$	$O(1)$
Searching	$O(n)$	$O(n)$	$O(1)$

Queue



1. A queue can be defined as an ordered list which enables insert operations to be performed at one end called **REAR** and delete operations to be performed at another end called **FRONT**.

2. Queue is referred to be as First In First Out list.

3. For example, people waiting in line for a rail ticket form a queue.

Applications of Queue

Due to the fact that queue performs actions on first in first out basis which is quite fair for the ordering of actions. There are various applications of queues discussed as below. Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.

1. Queues are used in asynchronous transfer of data (where data is not being transferred at the same rate between two processes) for eg. pipes, file IO, sockets.
2. Queues are used as buffers in most of the applications like MP3 media player, CD player, etc.
3. Queue are used to maintain the play list in media players in order to add and remove the songs from the play-list.
4. Queues are used in operating systems for handling interrupts.

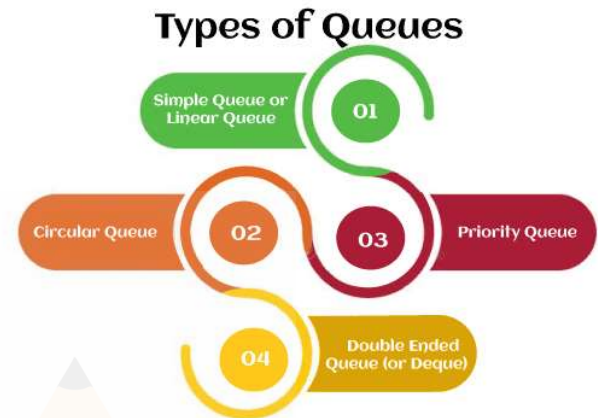
Complexity

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Addes	Search	Insertion	Deletion	Addes	Search	Insertion	Deletion	

	s				s				
Queue	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$

Types of Queue

There are four different types of queue that are listed as follows -



- Simple Queue or Linear Queue
- Circular Queue
- Priority Queue
- Double Ended Queue (or Deque)

Simple Queue or Linear Queue

- In Linear Queue, an insertion takes place from one end while the deletion occurs from another end.
- The end at which the insertion takes place is known as the rear end, and the end at which the deletion takes place is known as front end. It strictly follows the FIFO rule.

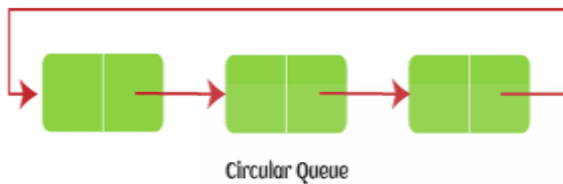


- The major drawback of using a linear Queue is that insertion is done only from the rear end.
- If the first three elements are deleted from the Queue, we cannot insert more elements even though the space is available in a Linear Queue.
- In this case, the linear Queue shows the overflow condition as the rear is pointing to the last element of the Queue.



Circular Queue

- In Circular Queue, all the nodes are represented as circular.
- It is similar to the linear Queue except that the last element of the queue is connected to the first element.
- It is also known as Ring Buffer, as all the ends are connected to another end.
- The representation of circular queue is shown in the below image -



- The drawback that occurs in a linear queue is overcome by using the circular queue.
- If the empty space is available in a circular queue, the new element can be added in an empty space by simply incrementing the value of rear.
- The main advantage of using the circular queue is better memory utilization.

Priority Queue

- It is a special type of queue in which the elements are arranged based on the priority.
- It is a special type of queue data structure in which every element has a priority associated with it.
- Suppose some elements occur with the same priority, they will be arranged according to the FIFO principle.
- The representation of priority queue is shown in the below image -



- Insertion in priority queue takes place based on the arrival, while deletion in the priority queue occurs based on the priority.

- Priority queue is mainly used to implement the CPU scheduling algorithms.

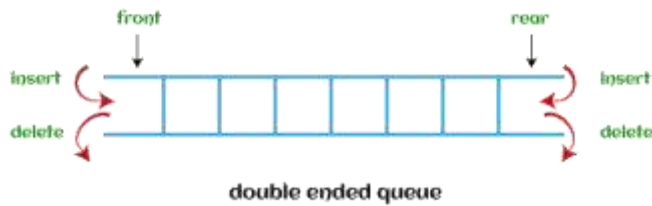
There are two types of priority queue that are discussed as follows -

- **Ascending priority queue** - In ascending priority queue, elements can be inserted in arbitrary order, but only smallest can be deleted first. Suppose an array with elements 7, 5, and 3 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 3, 5, 7.
- **Descending priority queue** - In descending priority queue, elements can be inserted in arbitrary order, but only the largest element can be deleted first. Suppose an array with elements 7, 3, and 5 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 7, 5, 3.

Deque (or, Double Ended Queue)

- In Deque or Double Ended Queue, insertion and deletion can be done from both ends of the queue either from the front or rear.
- It means that we can insert and delete elements from both front and rear ends of the queue.
- Deque can be used as a palindrome checker means that if we read the string from both ends, then the string would be the same.
- Deque can be used both as stack and queue as it allows the insertion and deletion operations on both ends.
- Deque can be considered as stack because stack follows the LIFO (Last In First Out) principle in which insertion and deletion both can be performed only from one end.
- And in deque, it is possible to perform both insertion and deletion from one end, and Deque does not follow the FIFO principle.

The representation of the deque is shown in the below image -

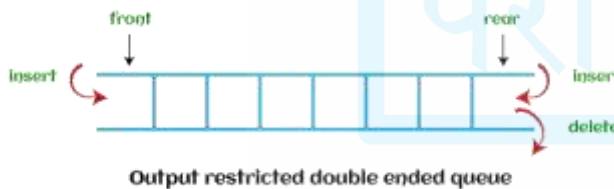


There are two types of deque that are discussed as follows -

- **Input restricted deque** - As the name implies, in input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



- **Output restricted deque** - As the name implies, in output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.



Now, let's see the operations performed on the queue.

Operations performed on queue

The fundamental operations that can be performed on queue are listed as follows -

- **Enqueue:** The Enqueue operation is used to insert the element at the rear end of the queue. It returns void.
- **Dequeue:** It performs the deletion from the front-end of the queue. It also returns the element which has been removed from the front-end. It returns an integer value.
- **Peek:** This is the third operation that returns the element, which is pointed by the front pointer in the queue but does not delete it.

- **Queue overflow (isfull):** It shows the overflow condition when the queue is completely full.
- **Queue underflow (isempty):** It shows the underflow condition when the Queue is empty, i.e., no elements are in the Queue.

What is a Stack?

- A Stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle.
- Stack has one end, whereas the Queue has two ends (**front and rear**).
- It contains only one pointer **top pointer** pointing to the topmost element of the stack.
- Whenever an element is added in the stack, it is added on the top of the stack, and the element can be deleted only from the stack. In other words, *a stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.*

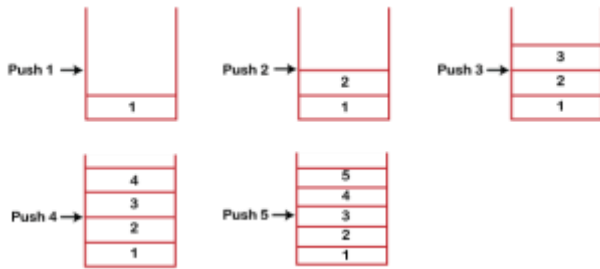
Some key points related to stack

- It is called as stack because it behaves like a real-world stack, piles of books, etc.
- A Stack is an abstract data type with a pre-defined capacity, which means that it can store the elements of a limited size.
- It is a data structure that follows some order to insert and delete the elements, and that order can be LIFO or FILO.

Working of Stack

Stack works on the LIFO pattern. As we can observe in the below figure there are five memory blocks in the stack; therefore, the size of the stack is 5.

- Suppose we want to store the elements in a stack and let's assume that stack is empty.
- We have taken the stack of size 5 as shown below in which we are pushing the elements one by one until the stack becomes full.



Since our stack is full as the size of the stack is 5. In the above cases, we can observe that it goes from the top to the bottom when we were entering the new element in the stack. The stack gets filled up from the bottom to the top.

When we perform the delete operation on the stack, there is only one way for entry and exit as the other end is closed. It follows the LIFO pattern, which means that the value entered first will be removed last. In the above case, the value 5 is entered first, so it will be removed only after the deletion of all the other elements.

Standard Stack Operations

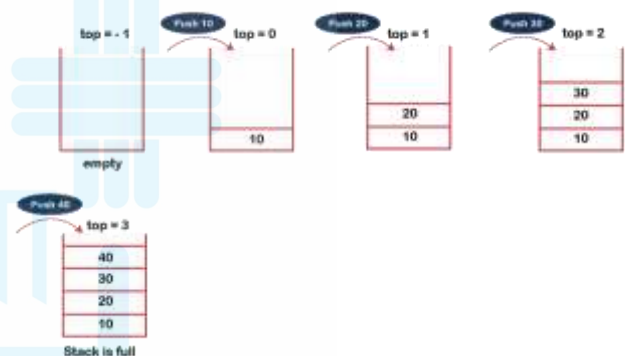
The following are some common operations implemented on the stack:

- **push():** When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- **pop():** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- **isEmpty():** It determines whether the stack is empty or not.
- **isFull():** It determines whether the stack is full or not.'
- **peek():** It returns the element at the given position.
- **count():** It returns the total number of elements available in a stack.
- **change():** It changes the element at the given position.
- **display():** It prints all the elements available in the stack.

PUSH operation

The steps involved in the PUSH operation is given below:

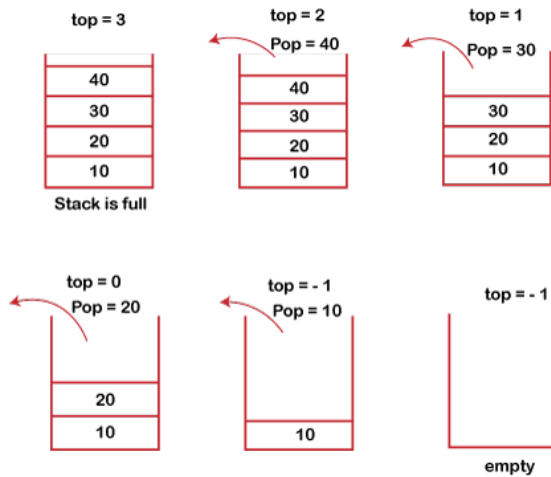
- Before inserting an element in a stack, we check whether the stack is full.
- If we try to insert the element in a stack, and the stack is full, then the **overflow** condition occurs.
- When we initialize a stack, we set the value of top as -1 to check that the stack is empty.
- When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **top=top+1**, and the element will be placed at the new position of the **top**.
- The elements will be inserted until we reach the **max** size of the stack.



POP operation

The steps involved in the POP operation is given below:

- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the element from the empty stack, then the **underflow** condition occurs.
- If the stack is not empty, we first access the element which is pointed by the **top**
- Once the pop operation is performed, the top is decremented by 1, i.e., **top=top-1**.



Applications of Stack

The following are the applications of the stack:

- **Balancing of symbols:** Stack is used for balancing a symbol. For example, we have the following program:

```
1. int main()
2. {
3.     cout<<"Hello";
4.     cout<<"javaTpoint";
5. }
```

As we know, each program has an *opening* and *closing* braces; when the opening braces come, we push the braces in a stack, and when the closing braces appear, we pop the opening braces from the stack. Therefore, the net value comes out to be zero. If any symbol is left in the stack, it means that some syntax occurs in a program.

- **String reversal:** Stack is also used for reversing a string. For example, we want to reverse a "javaTpoint" string, so we can achieve this with the help of a stack. First, we push all the characters of the string in a stack until we reach the null character. After pushing all the characters, we start taking out the character one by one until we reach the bottom of the stack.
- **UNDO/REDO:** It can also be used for performing UNDO/REDO operations. For example, we have an editor in which we write 'a', then 'b', and then 'c'; therefore, the text written in an editor is abc. So,

there are three states, a, ab, and abc, which are stored in a stack. There would be two stacks in which one stack shows UNDO state, and the other shows REDO state.

If we want to perform UNDO operation, and want to achieve 'ab' state, then we implement pop operation.

- **Recursion:** The recursion means that the function is calling itself again. To maintain the previous states, the compiler creates a system stack in which all the previous records of the function are maintained.
- **DFS(Depth First Search):** This search is implemented on a Graph, and Graph uses the stack data structure.
- **Backtracking:** Suppose we have to create a path to solve a maze problem. If we are moving in a particular path, and we realize that we come on the wrong way. In order to come at the beginning of the path to create a new path, we have to use the stack data structure.
- **Expression conversion:** Stack can also be used for expression conversion. This is one of the most important applications of stack. The list of the expression conversion is given below:
 - Infix to prefix
 - Infix to postfix
 - Prefix to infix
 - Prefix to postfix
 - Postfix to infix
- **Memory management:** The stack manages the memory. The memory is assigned in the contiguous memory blocks. The memory is known as stack memory as all the variables are assigned in a function call stack memory. The memory size assigned to the program is known to the compiler. When the function is created, all its variables are assigned in the stack memory. When the function



completed its execution, all the variables assigned in the stack are released.

Convert Infix to Postfix notation

What is infix notation?

When the operator is written in between the operands, then it is known as **infix notation**. Operand does not have to be always a constant or a variable; it can also be an expression itself.

For example,

$$(p + q) * (r + s)$$

Syntax of infix notation is given below:

<operand> <operator> <operand>

Priority

Operators	Symbols
Parenthesis	(), { }, []
Exponents	^
Multiplication and Division	*, /
Addition and Subtraction	+, -

Postfix Expression

The postfix expression is an expression in which the operator is written after the operands. For example, the postfix expression of infix notation $(2+3)$ can be written as $23+$.

Some key points regarding the postfix expression are:

- In postfix expression, operations are performed in the order in which they have written from left to right.
- It does not any require any parenthesis.
- We do not need to apply operator precedence rules and associativity rules.

Evaluation of postfix expression using stack.

- Scan the expression from left to right.
- If we encounter any operand in the expression, then we push the operand in the stack.
- When we encounter any operator in the expression, then we pop the corresponding operands from the stack.
- When we finish with the scanning of the expression, the final value remains in the stack.

Let's understand the evaluation of postfix expression using stack.

Example 1: Postfix expression: 2 3 4 * +

Input	Stack	
2 3 4 *	empty	Push 2
+		
3 4 * +	2	Push 3
4 * +	3 2	Push 4
* +	4 3 2	Pop 4 and 3, and perform $4*3 = 12$. Push 12 into the stack.
+	12 2	Pop 12 and 2 from the stack, and perform $12+2 = 14$. Push 14 into the stack.

The result of the above expression is 14.

Example 2: Postfix expression: 3 4 * 2 5 * +

Input	Stack	
3 4 * 2	empty	Push 3
5 * +		
4 * 2 5	3	Push 4
* +		
* 2 5 * +	4 3	Pop 3 and 4 from the stack and perform $3*4 = 12$. Push 12 into the stack.
2 5 * +	12	Push 2
5 * +	2 12	Push 5
* +	5 2 12	Pop 5 and 2 from the stack and perform $5*2 = 10$. Push 10 into the stack.
+	10 12	Pop 10 and 12 from the stack and perform $10+12 = 22$. Push 22 into the stack.

The result of the above expression is 22.

Conversion of infix to postfix

Here, we will use the stack data structure for the conversion of infix expression to postfix expression. Whenever an operator will encounter, we push operator into the stack. If we encounter an operand, then we append the operand to the expression.

Rules for the conversion from infix to postfix expression

1. Print the operand as they arrive.
2. If the stack is empty or contains a left parenthesis on top, push the incoming operator on to the stack.



- If the incoming symbol is '(', push it on to the stack.
- If the incoming symbol is ')', pop the stack and print the operators until the left parenthesis is found.
- If the incoming symbol has higher precedence than the top of the stack, push it on to the stack.
- If the incoming symbol has lower precedence than the top of the stack, pop and print the top of the stack. Then test the incoming operator against the new top of the stack.
- If the incoming operator has the same precedence with the top of the stack then use the associativity rules. If the associativity is from left to right then pop and print the top of the stack then push the incoming operator. If the associativity is from right to left then push the incoming operator.
- At the end of the expression, pop and print all the operators of the stack.

Let's understand through an example.

Infix expression: $K + L - M * N + (O \wedge P) * W / U / V * T + Q$

Input Expression	Stack	Postfix Expression
K		K
+	+	
L	+	K L
-	-	K L +
M	-	K L + M
*	- *	K L + M
N	- *	K L + M N
+	+	K L + M N * K L + M N * -
(	+(	K L + M N * -
O	+(	K L + M N * - O
^	+(^	K L + M N * - O
P	+(^	K L + M N * - O P
)	+	K L + M N * - O P ^
*	+	K L + M N * - O P ^
W	+	K L + M N * - O P ^ W
/	+/	K L + M N * - O P ^ W *
U	+/	K L + M N * - O P ^ W * U
/	+/	K L + M N * - O P ^ W * U /
V	+/	K L + M N * - O P ^ W * U / V
*	+	K L + M N * - O P ^ W * U / V /

T	+ *	KL+MN*-OP^W*U/V/T
+	+	KL+MN*-OP^W*U/V/T* KL+MN*-OP^W*U/V/T*+
Q	+	KL+MN*-OP^W*U/V/T*Q
		KL+MN*- OP^W*U/V/T*+Q+

The final postfix expression of infix expression $(K + L - M * N + (O \wedge P) * W / U / V * T + Q)$ is $KL+MN*-OP^W*U/V/T*+Q+$.

Convert infix to prefix notation

What is Prefix notation?

A prefix notation is another form of expression but it does not require other information such as precedence and associativity, whereas an infix notation requires information of precedence and associativity. It is also known as **polish notation**. In prefix notation, an operator comes before the operands. The syntax of prefix notation is given below:

<operator> <operand> <operand>

For example, if the infix expression is $5+1$, then the prefix expression corresponding to the infix expression is:

a * b + c

***ab+c**

+*abc (Prefix expression)

Conversion of Infix to Prefix using Stack

$K + L - M * N + (O \wedge P) * W / U / V * T + Q$

If we are converting the expression from infix to prefix, we need first to reverse the expression.

The Reverse expression would be:

$Q + T * V / U / W *) P \wedge O (+ N * M - L + K$

To obtain the prefix expression, we have created a table that consists of three columns, i.e., input expression, stack, and prefix expression. When we encounter any symbol, we simply add it into the prefix expression. If we encounter the operator, we will push it into the stack.

Input expression	Stack	Prefix expression
Q		Q
+	+	Q



T	+	QT
*	++	QT
V	++	QTV
/	++/	QTV
U	++/	QTVU
/	++//	QTVU
W	++//	QTVUW
*	++//*	QTVUW
)	++//*)	QTVUW
P	++//*)	QTVUWP
^	++//*)^	QTVUWP
O	++//*)^	QTVUWPO
(	++//*	QTVUWPO^
+	++	QTVUWPO^*//*
N	++	QTVUWPO^*//*N
*	++*	QTVUWPO^*//*N
M	++*	QTVUWPO^*//*NM
-	++-	QTVUWPO^*//*NM*
L	++-	QTVUWPO^*//*NM*L
+	++-+	QTVUWPO^*//*NM*L
K	++-+	QTVUWPO^*//*NM*LK
		QTVUWPO^*//*NM*LK+-
		++

The above expression, i.e., QTVUWPO^*//*NM*LK+-
++, is not a final expression. We need to reverse this
expression to obtain the prefix expression. ++-
KL*MN*//^*OPWUUVTQ

Rules for the conversion of infix to prefix expression:

- First, reverse the infix expression given in the problem.
- Scan the expression from left to right.
- Whenever the operands arrive, print them.
- If the operator arrives and the stack is found to be empty, then simply push the operator into the stack.
- If the incoming operator has higher precedence than the TOP of the stack, push the incoming operator into the stack.
- If the incoming operator has the same precedence with a TOP of the stack, push the incoming operator into the stack.

- If the incoming operator has lower precedence than the TOP of the stack, pop, and print the top of the stack. Test the incoming operator against the top of the stack again and pop the operator from the stack till it finds the operator of a lower precedence or same precedence.
- If the incoming operator has the same precedence with the top of the stack and the incoming operator is ^, then pop the top of the stack till the condition is true. If the condition is not true, push the ^ operator.
- When we reach the end of the expression, pop, and print all the operators from the top of the stack.
- If the operator is ')', then push it into the stack.
- If the operator is '(', then pop all the operators from the stack till it finds) opening bracket in the stack.
- If the top of the stack is ')', push the operator on the stack.
- At the end, reverse the output.

Conversion of Prefix to Postfix expression

For example:

If the infix expression is given as: A + B * C

As we know that the multiplication operator * has a higher precedence than the addition operator. First, multiplication operator will move before operand B shown as below:

A + * B C

Once the multiplication operator is moved before 'B' operand, addition operator will move before the operand 'A' shown as below:

+ A * B C

Evaluation of Prefix Expression using Stack

Step 1: Initialize a pointer 'S' pointing to the end of the expression.

Step 2: If the symbol pointed by 'S' is an operand then push it into the stack.

Step 3: If the symbol pointed by 'S' is an operator then pop two operands from the stack. Perform the operation on these two operands and stores the result into the stack.

Step 4: Decrement the pointer 'S' by 1 and move to step 2 as long as the symbols left in the expression.



Step 5: The final result is stored at the top of the stack and return it.

Step 6: End

Let's understand the evaluation of prefix expression through an example.

Expression: +, -, *, 2, 2, /, 16, 8, 5

First, we will reverse the expression given above.

Expression: 5, 8, 16, /, 2, 2, *, -, +

We will use the stack data structure to evaluate the prefix expression.

Symbol Scanned	Stack
5	5
8	5, 8
16	5, 8, 16
/	5, 2
2	5, 2, 2
2	5, 2, 2, 2
*	5, 2, 4
-	5, 2
+	7

The final result of the above expression is 7.

Conversion of Prefix to Postfix Expression

Here, we will see the conversion of prefix to postfix expression using a stack data structure.

Rules for prefix to postfix expression using stack data structure:

- Scan the prefix expression from right to left, i.e., reverse.
- If the incoming symbol is an operand then push it into the stack.
- If the incoming symbol is an operator then pop two operands from the stack. Once the operands are popped out from the stack, we add the incoming symbol after the operands. When the operator is added after the operands, then the expression is pushed back into the stack.
- Once the whole expression is scanned, pop and print the postfix expression from the stack.

Let's understand the conversion of Prefix to Postfix expression using Stack through an example.

If the expression is: * - A / B C - / A K L

Symbols to be scanned	Action	Stack	Description
L	Push L into the stack	L	
K	Push K into the stack	L, K	
A	Push A into the stack	L, K, A	
/	Pop A from the stack Pop K from the stack Push A K / into the stack	L, A K /	Pop two operands from the stack, i.e., A and K. Add '/' operator after K operand, i.e., AK/. Push AK/ into the stack.
-	Pop A K / and L from the stack. Push (A K / L -) into the stack	A K / L -	Pop two operands from the stack, i.e., AK/ and L. Add '-' operator after 'L' operand.
C	Push C into the stack	AK/L-, C	
B	Push B into the stack	AK/L-, C, B	
/	Pop B and C from the stack. Push BC/ into the stack.	AK/L-, BC/	Pop two operands from the stack, i.e., B and C. Add '/' operator after C operator, i.e., BC/. Push BC/ into the stack.
A	Push A into the stack	AK/L-, BC/, A	
-	Pop BC/ and A from the stack. Push	AK/L-, ABC/-	Pop two operands from the stack, i.e., A



	ABC/- into the stack.		and BC/. Add '-' operator after '/'.
*	Pop ABC/- and AK/L- from the stack. Push ABC/AK/L-* into the stack.	ABC/- AK/L- *	Pop two operands from the stack, i.e., ABC/-, and AK/L-. Add '*' operator after L and '-' operator, i.e., ABC/- AK/L-.*.

Conversion of Postfix to Prefix expression

Conversion of Postfix to Prefix expression using Stack

The following are the steps used to convert postfix to prefix expression using stack:

- Scan the postfix expression from left to right.
- If the element is an operand, then push it into the stack.
- If the element is an operator, then pop two operands from the stack.

Create an expression by concatenating two operands and adding operator before the operands.

Push the result back to the stack.

- Repeat the above steps until we reach the end of the postfix expression.
- **Let's understand the conversion of postfix to prefix expression using stack.**
- **If the Postfix expression is given as:**
- **AB + CD - ***

Symbol Scanned	Action	Stack	Description
A	Push A into the stack	A	
B	Push B into the stack	AB	
+	Pop B from the	+AB	Pop two operands from the stack, i.e., A and B. Add '+' operator before

	stack Pop A from the stack Push +AB into the stack.		the operands AB, i.e., +AB.
C	Push C into the stack	+ABC	
D	Push D into the stack	+ABCD	
-	Pop D from the stack. Pop C from the stack. Push - CD into the stack	+AB - CD	Pop two operands from the stack, i.e., D and C. Add '-' operator before the operands CD, i.e., - CD.
*	Pop - CD from the stack. Pop +AB from the stack. Push *+AB - CD	*+AB - CD	Pop two operands from the stack, i.e., -CD and +AB. Add '*' operator before +AB then the expression would become *+AB-CD.



	into the stack.		
--	-----------------------	--	--

- The prefix expression of the above postfix expression is $*+AB-CD$.

Conversion of Postfix expression to Infix expression

Postfix to infix conversion example with dry run

(1) Postfix expression

a b c / - a d / e - *

↑
'a' is an operand push it into the stack.

Stack

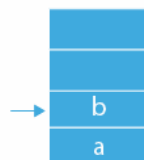


(2) Postfix expression

a b c / - a d / e - *

↑
'b' is an operand push it into the stack.

Stack

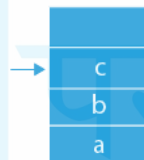


(3) Postfix expression

a b c / - a d / e - *

↑
'c' is an operand. So, push it into the stack.

Stack



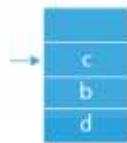
(4) Postfix expression

a b c / - a d / e - *

↑
'/' is an operator. Pop two expressions from the stack.

Operand 1 = c
Operand 2 = b
Operator = /

Stack



|| Pop

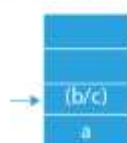


Push

"(Operand 2, Operator, Operand 1)"

=> (b/c)
into the stack.

|| Push



(5) Postfix expression

a b c / - a d / e - *

↑
'-' is an operator. So, operand 1 = (b/c)

Operand 2 = a

push (operand 2 operator operand 1)
i.e. (a - (b/c)) into the stack.

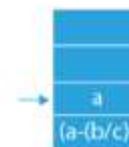


(6) Postfix expression

a b c / - a d / e - *

↑
'a' is an operand. So, push it into the stack.

Stack

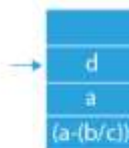


(7) Postfix expression

a b c / - a d / e - *

↑
'd' is an operand. So, push it into the stack.

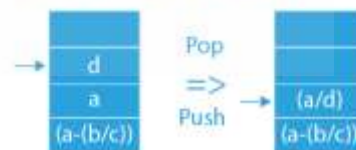
Stack



(8) Postfix expression

a b c / - a d / e - *

↑
'/' is an operator.
Operand 1 = d
Operand 2 = a
Push, (operand 2 operator operand 1) i.e (a/d)



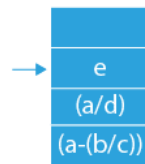


(9) Postfix expression

Stack

 $a\ b\ c\ /\ -\ a\ d\ /\ e\ -\ *$

'e' is an operand. So, push it into the stack.



(10) Postfix expression

 $a\ b\ c\ /\ -\ a\ d\ /\ e\ -\ *$

Operator = -

Operand 1 = e

Operand 2 = (a/d)

Push, (operand 2 operator operand 1) i.e ((a/d)-e)



(11) Postfix expression

 $a\ b\ c\ /\ -\ a\ d\ /\ e\ -\ *$

Operator = *

Operand 1 = ((a/d)-e)

Operand 2 = (a-(b/c))

Push, (operand 2 operator operand 1) i.e ((a-(b/c))*((a/d)-e)

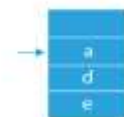


(4) Prefix expression

Stack

 $*\ -\ a\ /\ b\ c\ -\ /\ a\ d\ e$

'/' is an operator. Pop two expressions from the stack.



Operand 1 = a

Operand 2 = d

Operand 3 = /

Pop



Calculate the expression

 $*(\text{Operand 1} + \text{Operator} + \text{Operand 2})$ $\Rightarrow (a/d)$

NOTE: + Indicate concatenation.



(5) Prefix expression

 $*\ -\ a\ /\ b\ c\ -\ /\ a\ d\ e$

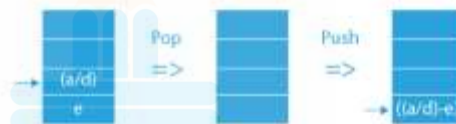
'-' is an operator. So, operand 1 = (a/d)

Operand 1 = (a/d)

Operand 2 = e

Operand = -

push (+ operand 1 + operator + operand 2 +) into the stack



(6) Prefix expression

Stack

 $*\ -\ a\ /\ b\ c\ -\ /\ a\ d\ e$

'c' is an operand. So, push it into the stack.

Conversion of Prefix expression to Infix expression

(1) Prefix expression

Stack

 $*\ -\ a\ /\ b\ c\ -\ /\ a\ d\ e$

'e' is an operand push it into the stack.



(2) Prefix expression

Stack

 $*\ -\ a\ /\ b\ c\ -\ /\ a\ d\ e$

'd' is an operand push it into the stack.



(3) Prefix expression

Stack

 $*\ -\ a\ /\ b\ c\ -\ /\ a\ d\ e$

'a' is an operand. So, push it into the stack.



(7) Prefix expression

Stack

 $*\ -\ a\ /\ b\ c\ -\ /\ a\ d\ e$

'b' is an operand. So, push it into the stack.



(8) Prefix expression

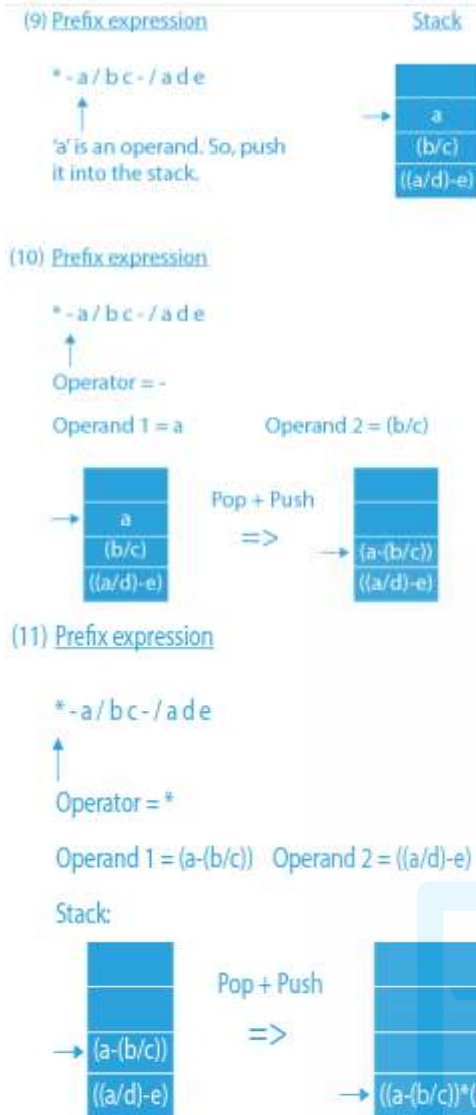
 $*\ -\ a\ /\ b\ c\ -\ /\ a\ d\ e$

'/' is an operator.

Operand 1 = b

Operand 2 = c





Tree Data Structure

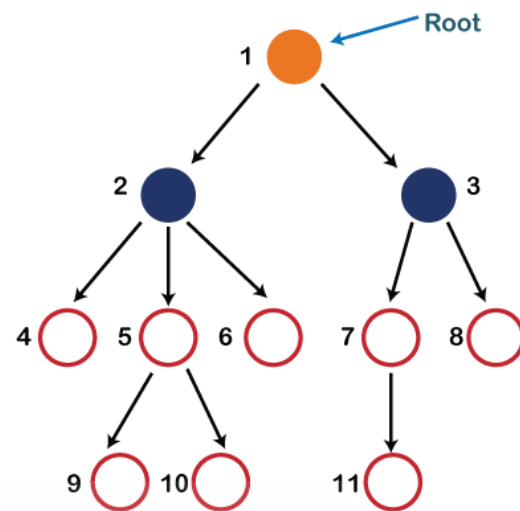
We read the linear data structures like an array, linked list, stack and queue in which all the elements are arranged in a sequential manner. The different data structures are used for different kinds of data.

A *tree* is also one of the data structures that represent hierarchical data.

Some basic terms used in Tree data structure.

Let's consider the tree structure, which is shown below:

Introduction to Trees



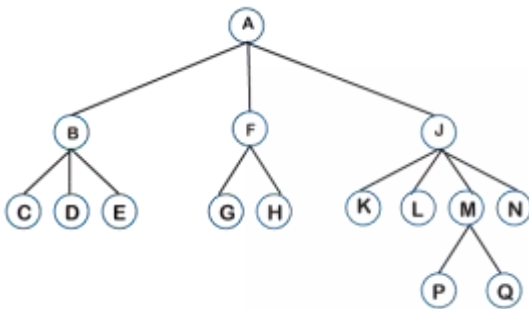
In the above structure, each node is labeled with some number. Each arrow shown in the above figure is known as a **link** between the two nodes.

- **Root:** The root node is the topmost node in the tree hierarchy. In other words, the root node is the one that doesn't have any parent. In the above structure, node numbered 1 is **the root node of the tree**. If a node is directly linked to some other node, it would be called a parent-child relationship.
- **Child node:** If the node is a descendant of any node, then the node is known as a child node.
- **Parent:** If the node contains any sub-node, then that node is said to be the parent of that sub-node.
- **Sibling:** The nodes that have the same parent are known as siblings.
- **Leaf Node:-** The node of the tree, which doesn't have any child node, is called a leaf node. A leaf node is the bottom-most node of the tree. There can be any number of leaf nodes present in a general tree. Leaf nodes can also be called external nodes.
- **Internal nodes:** A node has at least one child node known as an **internal**.
- **Ancestor node:-** An ancestor of a node is any predecessor node on a path from the root to that node. The root node doesn't have any ancestors. In the tree shown in the above image, nodes 1, 2, and 5 are the ancestors of node 10.
- **Descendant:** The immediate successor of the given node is known as a descendant of a node. In the above figure, 10 is the descendant of node 5.

Types of Tree data structure

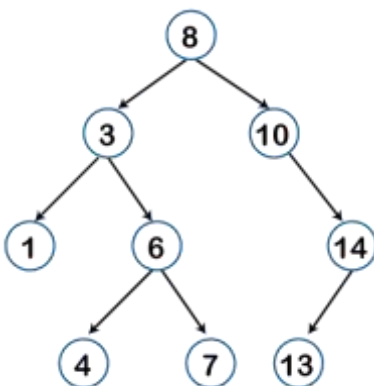
The following are the types of a tree data structure:

- **General tree:** The general tree is one of the types of tree data structure. In the general tree, a node can have either 0 or maximum n number of nodes. There is no restriction imposed on the degree of the node (the number of nodes that a node can contain). The topmost node in a general tree is known as a root node. The children of the parent node are known as *subtrees*.



There can be n number of subtrees in a general tree. In the general tree, the subtrees are unordered as the nodes in the subtree cannot be ordered. Every non-empty tree has a downward edge, and these edges are connected to the nodes known as *child nodes*. The root node is labeled with level 0. The nodes that have the same parent are known as *siblings*.

- **Binary tree:** Here, binary name itself suggests two numbers, i.e., 0 and 1. In a binary tree, each node in a tree can have utmost two child nodes. Here, utmost means whether the node has 0 nodes, 1 node or 2 nodes.



Types of Binary Tree

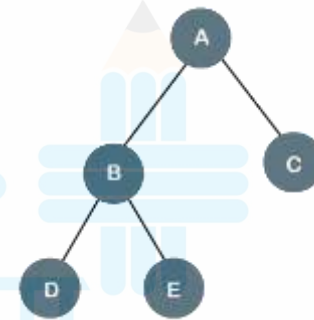
There are four types of Binary tree:

- Full/ proper/ strict Binary tree
- Complete Binary tree
- Perfect Binary tree
- Degenerate Binary tree
- Balanced Binary tree

➤ Full/ proper/ strict Binary tree

The full binary tree is also known as a strict binary tree. The tree can only be considered as the full binary tree if each node must contain either 0 or 2 children. The full binary tree can also be defined as the tree in which each node must contain 2 children except the leaf nodes.

Let's look at the simple example of the Full Binary tree.

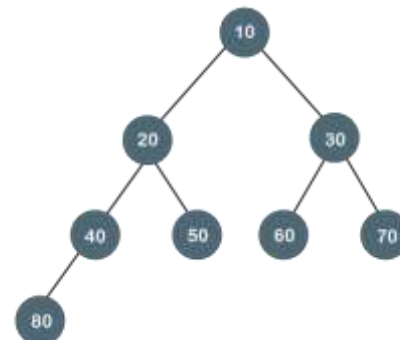


In the above tree, we can observe that each node is either containing zero or two children; therefore, it is a Full Binary tree.

➤ Complete Binary Tree

The complete binary tree is a tree in which all the nodes are completely filled except the last level. In the last level, all the nodes must be as left as possible. In a complete binary tree, the nodes should be added from the left.

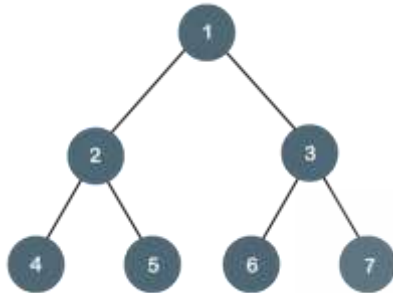
Let's create a complete binary tree.



The above tree is a complete binary tree because all the nodes are completely filled, and all the nodes in the last level are added at the left first.

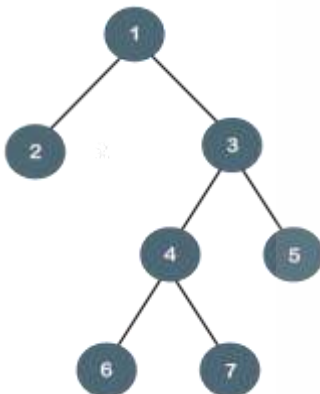
➤ Perfect Binary Tree

A tree is a perfect binary tree if all the internal nodes have 2 children, and all the leaf nodes are at the same level.



Let's look at a simple example of a perfect binary tree.

The below tree is not a perfect binary tree because all the leaf nodes are not at the same level.

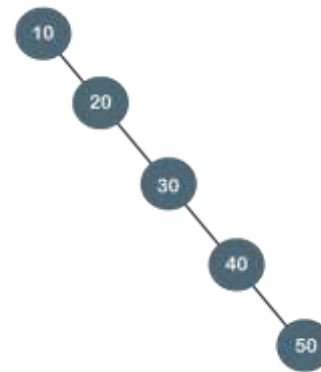


Note: All the perfect binary trees are the complete binary trees as well as the full binary tree, but vice versa is not true, i.e., all complete binary trees and full binary trees are the perfect binary trees.

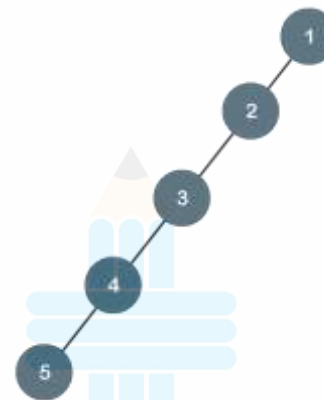
➤ Degenerate Binary Tree

The degenerate binary tree is a tree in which all the internal nodes have only one children.

Let's understand the Degenerate binary tree through examples.



The above tree is a degenerate binary tree because all the nodes have only one child. It is also known as a right-skewed tree as all the nodes have a right child only.

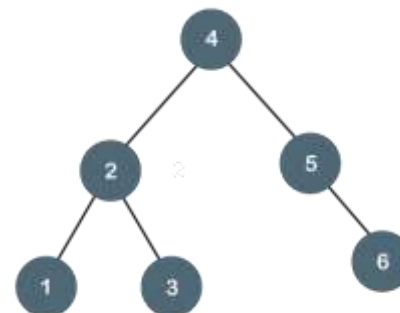


The above tree is also a degenerate binary tree because all the nodes have only one child. It is also known as a left-skewed tree as all the nodes have a left child only.

➤ Balanced Binary Tree

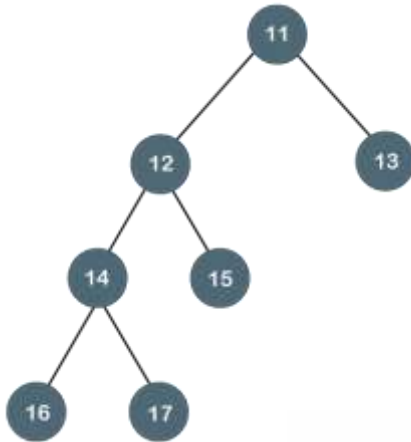
The balanced binary tree is a tree in which both the left and right trees differ by at most 1. For example, AVL and Red-Black trees are balanced binary tree.

Let's understand the balanced binary tree through examples.





The above tree is a balanced binary tree because the difference between the left subtree and right subtree is zero.

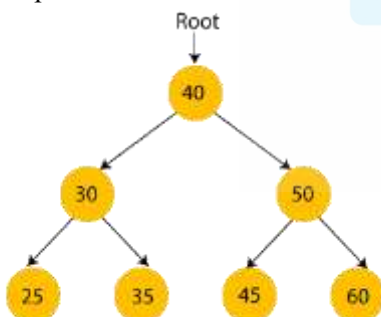


The above tree is not a balanced binary tree because the difference between the left subtree and the right subtree is greater than 1.

Binary Search tree

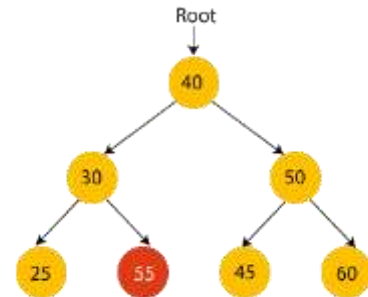
A binary search tree follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

Let's understand the concept of Binary search tree with an example.



- In the above figure, we can observe that the root node is 40, and all the nodes of the left subtree are smaller than the root node, and all the nodes of the right subtree are greater than the root node.
- Similarly, we can see the left child of root node is greater than its left child and smaller than its right child. So, it also satisfies the property of binary search tree. Therefore, we can say that the tree in the above image is a binary search tree.

- Suppose if we change the value of node 35 to 55 in the above tree, check whether the tree will be binary search tree or not.



In the above tree, the value of root node is 40, which is greater than its left child 30 but smaller than right child of 30, i.e., 55. So, the above tree does not satisfy the property of Binary search tree. Therefore, the above tree is not a binary search tree.

Advantages of Binary search tree

- Searching an element in the Binary search tree is easy as we always have a hint that which subtree has the desired element.
- As compared to array and linked lists, insertion and deletion operations are faster in BST.

Example of creating a binary search tree

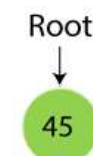
Now, let's see the creation of binary search tree using an example.

Suppose the data elements are - 45, 15, 79, 90, 10, 55, 12, 20, 50

- First, we have to insert 45 into the tree as the root of the tree.
- Then, read the next element; if it is smaller than the root node, insert it as the root of the left subtree, and move to the next element.
- Otherwise, if the element is larger than the root node, then insert it as the root of the right subtree.

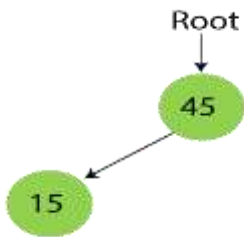
Now, let's see the process of creating the Binary search tree using the given data element. The process of creating the BST is shown below -

Step 1 - Insert 45.

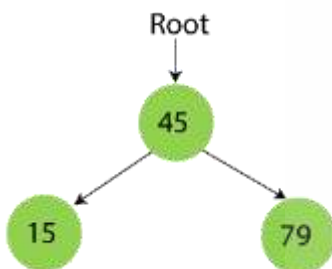


**Step 2 - Insert 15.**

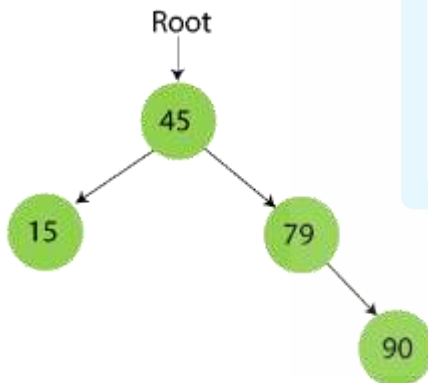
As 15 is smaller than 45, so insert it as the root node of the left subtree.

**Step 3 - Insert 79.**

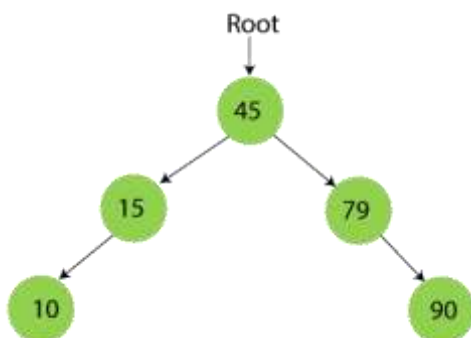
As 79 is greater than 45, so insert it as the root node of the right subtree.

**Step 4 - Insert 90.**

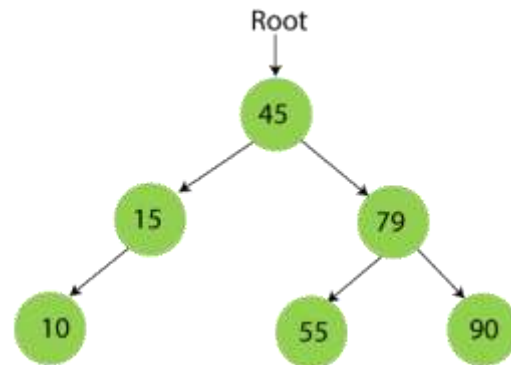
90 is greater than 45 and 79, so it will be inserted as the right subtree of 79.

**Step 5 - Insert 10.**

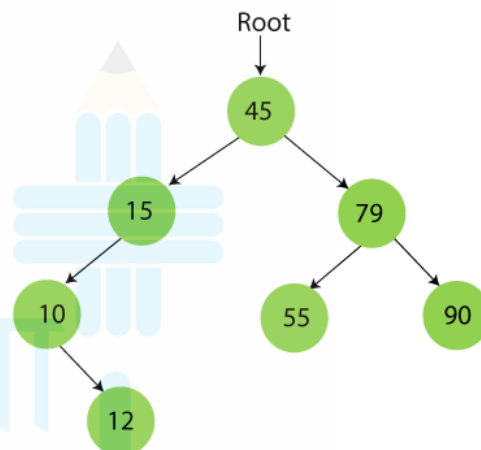
10 is smaller than 45 and 15, so it will be inserted as a left subtree of 15.

**Step 6 - Insert 55.**

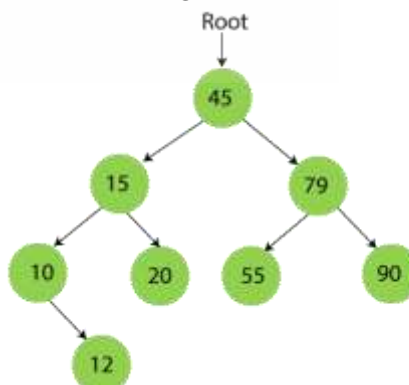
55 is larger than 45 and smaller than 79, so it will be inserted as the left subtree of 79.

**Step 7 - Insert 12.**

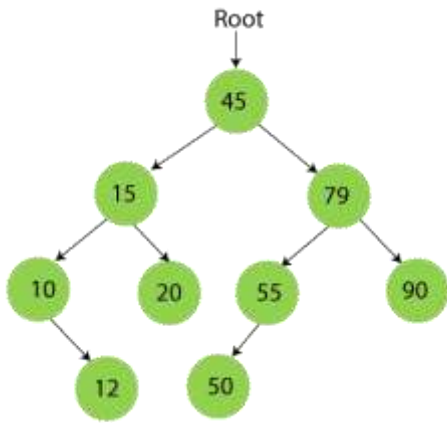
12 is smaller than 45 and 15 but greater than 10, so it will be inserted as the right subtree of 10.

**Step 8 - Insert 20.**

20 is smaller than 45 but greater than 15, so it will be inserted as the right subtree of 15.

**Step 9 - Insert 50.**

50 is greater than 45 but smaller than 79 and 55. So, it will be inserted as a left subtree of 55.



Now, the creation of binary search tree is completed. After that, let's move towards the operations that can be performed on Binary search tree.

The complexity of the Binary Search tree

Let's see the time and space complexity of the Binary search tree. We will see the time complexity for insertion, deletion, and searching operations in best case, average case, and worst case.

1. Time Complexity

Operations	Best case time complexity	Average case time complexity	Worst case time complexity
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(\log n)$	$O(n)$
Search	$O(\log n)$	$O(\log n)$	$O(n)$

Where 'n' is the number of nodes in the given tree.

2. Space Complexity

Operations	Space complexity
Insertion	$O(n)$
Deletion	$O(n)$
Search	$O(n)$

AVL Tree

- AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors.
- AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.

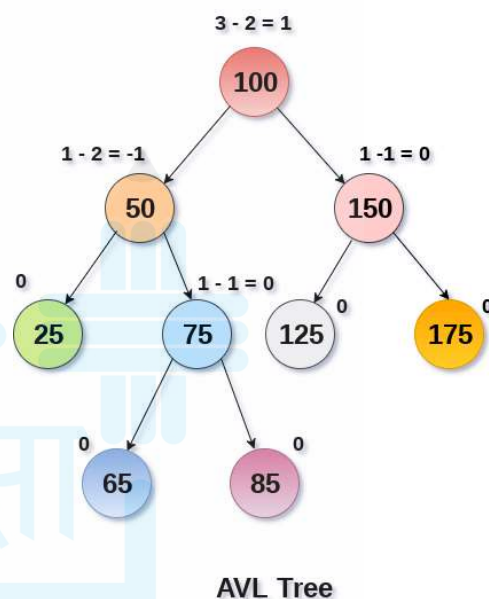
- Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

Balance Factor (k) = height (left(k)) - height (right(k))

If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

An AVL tree is given in the following figure. We can see that, balance factor associated with each node is in between -1 and +1. therefore, it is an example of AVL tree.



Complexity

Algorithm	Average case	Worst case
Space	$o(n)$	$o(n)$
Search	$o(\log n)$	$o(\log n)$
Insert	$o(\log n)$	$o(\log n)$
Delete	$o(\log n)$	$o(\log n)$

B Tree

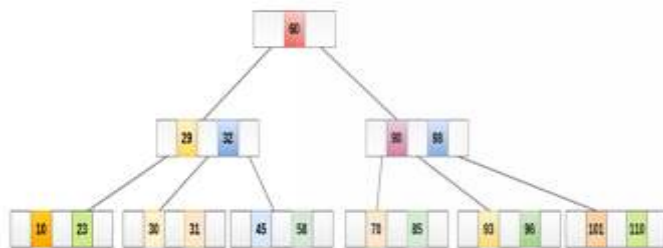
B Tree is a specialized m-way tree that can be widely used for disk access. A B-Tree of order m can have at most m-1 keys and m children. One of the main reason of using B tree is its capability to store large number of keys in a single node and large key values by keeping the height of the tree relatively small.

A B tree of order m contains all the properties of an M way tree. In addition, it contains the following properties.

1. Every node in a B-Tree contains at most m children.
2. Every node in a B-Tree except the root node and the leaf node contain at least $m/2$ children.
3. The root nodes must have at least 2 nodes.
4. All leaf nodes must be at the same level.

It is not necessary that, all the nodes contain the same number of children but, each node must have $m/2$ number of nodes.

A B tree of order 4 is shown in the following image.



While performing some operations on B Tree, any property of B Tree may violate such as number of minimum children a node can have. To maintain the properties of B Tree, the tree may split or join.

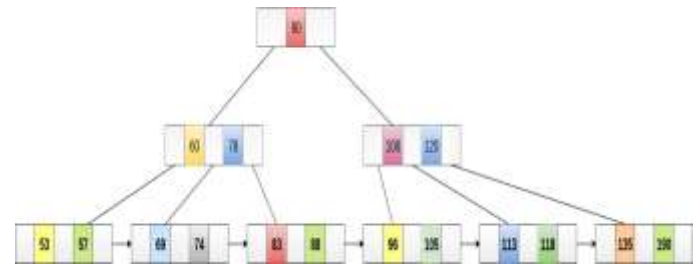
B+ Tree

B+ Tree is an extension of B Tree which allows efficient insertion, deletion and search operations.

In B Tree, Keys and records both can be stored in the internal as well as leaf nodes. Whereas, in B+ tree, records (data) can only be stored on the leaf nodes while internal nodes can only store the key values.

- The leaf nodes of a B+ tree are linked together in the form of a singly linked lists to make the search queries more efficient.
- B+ Tree are used to store the large amount of data which can not be stored in the main memory. Due to the fact that, size of main memory is always limited, the internal nodes (keys to access records) of the B+ tree are stored in the main memory whereas, leaf nodes are stored in the secondary memory.

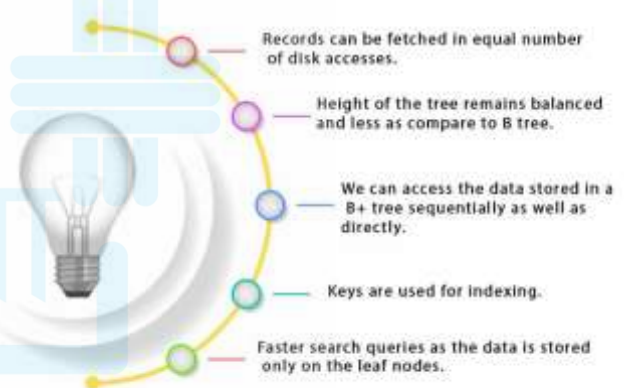
re often called index nodes. A B+ tree of order 3 is shown in the following figure.



Advantages of B+ Tree

1. Records can be fetched in equal number of disk accesses.
2. Height of the tree remains balanced and less as compare to B tree.
3. We can access the data stored in a B+ tree sequentially as well as directly.
4. Keys are used for indexing.
5. Faster search queries as the data is stored only on the leaf nodes.

Advantages of B+ Tree



Inorder Traversal

In this article, we will discuss the inorder traversal in data structure.

If we want to traverse the nodes in ascending order, then we use the inorder traversal. Following are the steps required for the inorder traversal:

- Visit all the nodes in the left subtree
- Visit the root node
- Visit all the nodes in the right subtree

Linear data structures such as stack, array, queue, etc., only have one way to traverse the data. But in hierarchical data structures such as **tree**, there are multiple ways to traverse the data. Here we will discuss another way to traverse the tree data structure, i.e., inorder traversal.

There are two approaches used for the inorder traversal:

- Inorder traversal using Recursion
- Inorder traversal using an Iterative method

An inorder traversal technique follows the **Left Root Right** policy. Here, Left Root Right means that the left subtree of the root node is traversed first, then the root node, and then the right subtree of the root node is traversed. Here, inorder name itself suggests that the root node comes in between the left and the right subtrees.

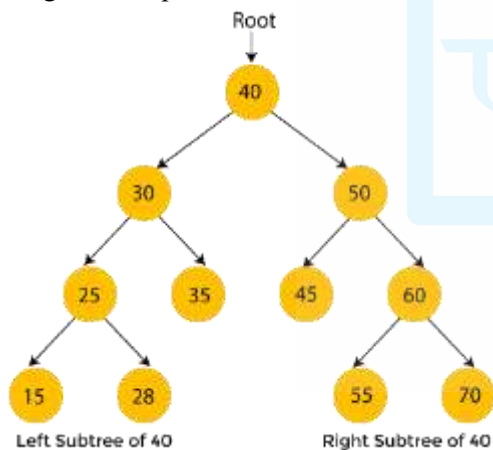
We will discuss the inorder traversal using both recursive and iterative techniques. Let's first start with inorder traversal using recursion.

Inorder traversal using recursion

1. Step 1: Recursively traverse the left subtree
2. Step 2: Now, visit the root
3. Step 3: Traverse the right subtree recursively

Example of inorder traversal

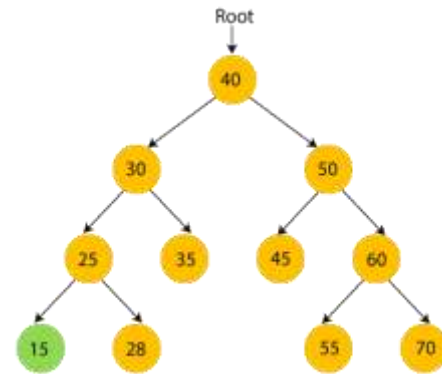
Now, let's see an example of inorder traversal. It will be easier to understand the procedure of inorder traversal using an example.



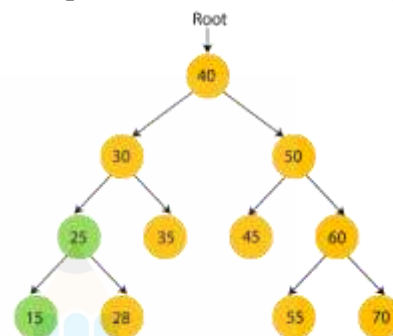
The nodes with yellow color are not visited yet. Now, we will traverse the nodes of the above tree using inorder traversal.

- Here, 40 is the root node. We move to the left subtree of 40, that is 30, and it also has subtree 25, so we again move to the left subtree of 25 that is 15. Here, 15 has no subtree, so **print 15** and move

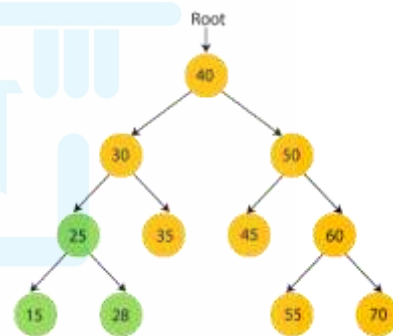
towards its parent node, 25.



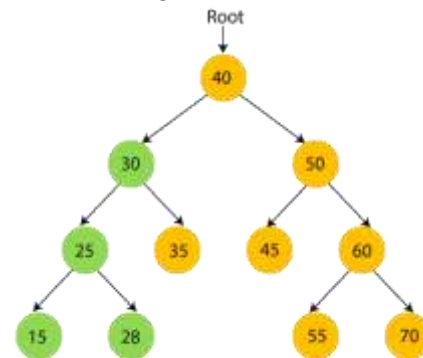
- Now, **print 25** and move to the right subtree of 25.



- Now, **print 28** and move to the root node of 25 that is 30.

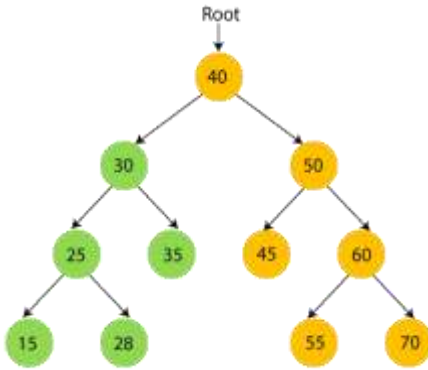


- So, left subtree of 30 is visited. Now, **print 30** and move to the right child of 30.

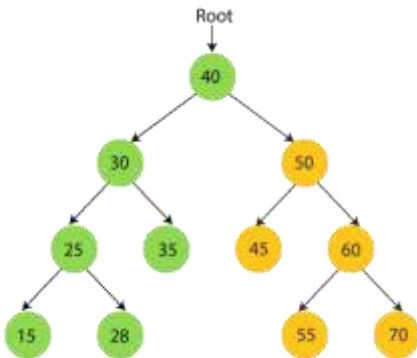




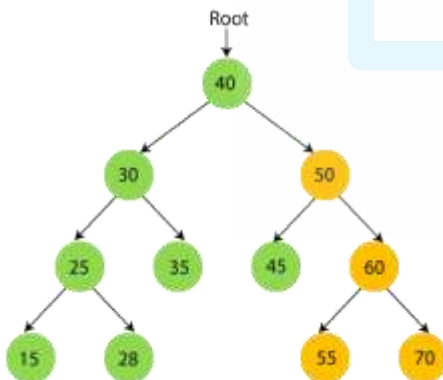
- Now, **print 35** and move to the root node of 30.



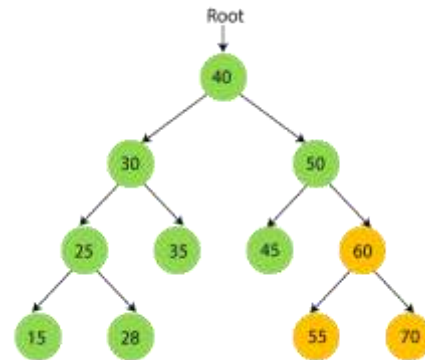
- Now, **print root node 40** and move to its right subtree.



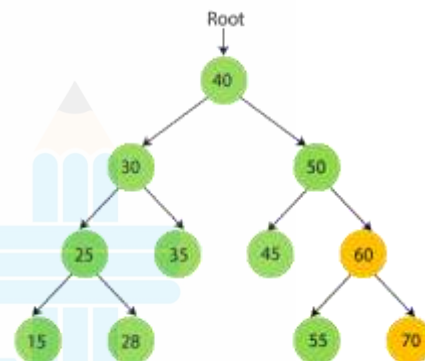
- Now recursively traverse the right subtree of 40 that is 50.
50 have subtree so first traverse the left subtree of 50 that is 45. 45 has no children, so **print 45** and move to its root node.



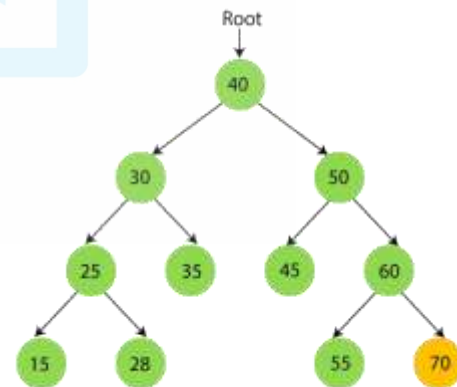
- Now **print 50** and move to the right subtree of 50 that is 60.



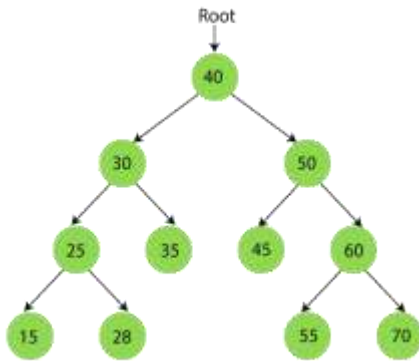
- Now recursively traverse the right subtree of 50 that is 60. 60 have subtree so first traverse the left subtree of 60 that is 55. 55 has no children, so **print 55** and move to its root node.



- Now **print 60** and move to the right subtree of 60 that is 70.



- Now **print 70**.



After the completion of inorder traversal, the final output is -
{15, 25, 28, 30, 35, 40, 45, 50, 55, 60, 70}

Complexity of Inorder traversal

The time complexity of Inorder traversal is $O(n)$, where 'n' is the size of binary tree.

Whereas, the space complexity of inorder traversal is $O(1)$, if we do not consider the stack size for function calls. Otherwise, the space complexity of inorder traversal is $O(h)$, where 'h' is the height of tree.

Preorder Traversal

In this article, we will discuss the preorder traversal in data structure. Linear data structures such as stack, array, queue, etc., only have one way to traverse the data. But in a hierarchical data structure such as **tree**, there are multiple ways to traverse the data.

In preorder traversal, first, root node is visited, then left sub-tree and after that right sub-tree is visited. The process of preorder traversal can be represented as –

1. root → left → right

Root node is always traversed first in preorder traversal, while it is the last item of postorder traversal. Preorder traversal is used to get the prefix expression of a tree.

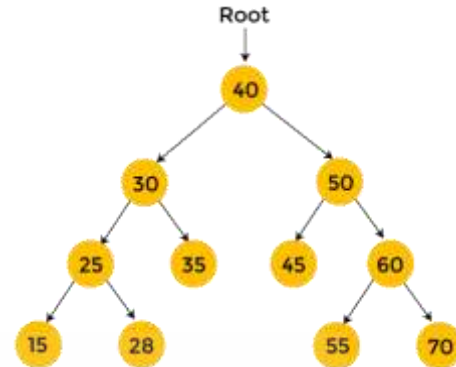
The steps to perform the preorder traversal are listed as follows -

- First, visit the root node.
- Then, visit the left subtree.
- At last, visit the right subtree.

The preorder traversal technique follows the **Root Left Right** policy. The name preorder itself suggests that the root node would be traversed first.

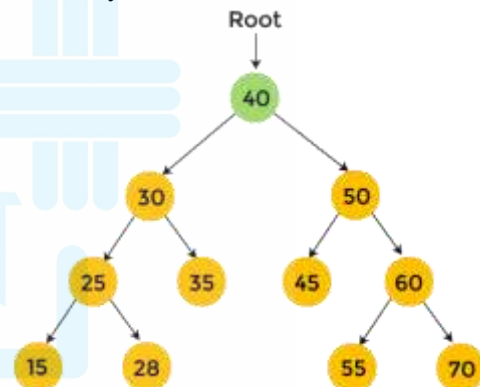
Example of preorder traversal

Now, let's see an example of preorder traversal. It will be easier to understand the process of preorder traversal using an example.

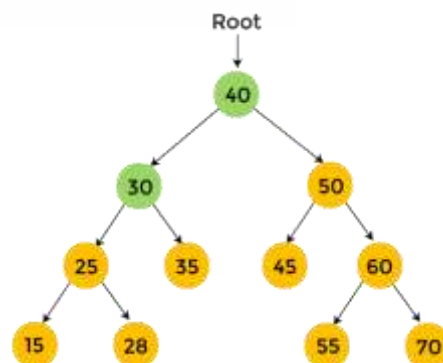


The nodes with yellow color are not visited yet. Now, we will traverse the nodes of the above tree using preorder traversal.

- Start with the root node 40. First, **print 40** and then recursively traverse the left subtree.

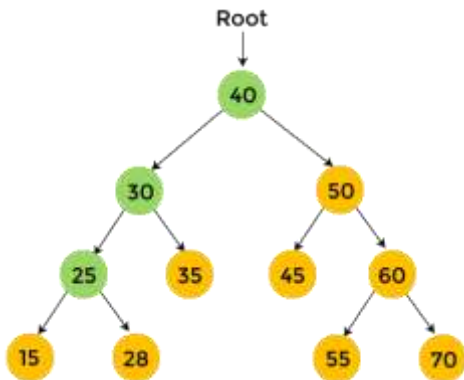


- Now, move to the left subtree. For left subtree, the root node is 30. **Print 30**, and move towards the left subtree of 30.

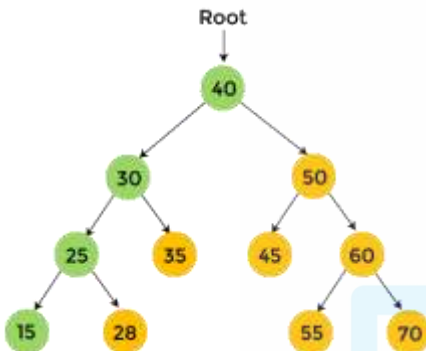




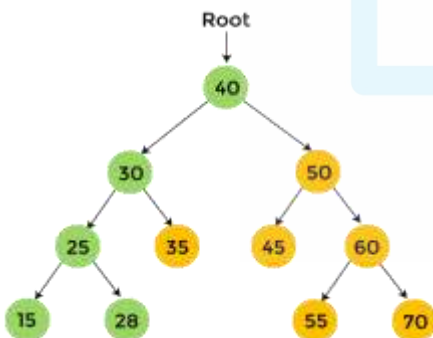
- In left subtree of 30, there is an element 25, so **print 25**, and traverse the left subtree of 25.



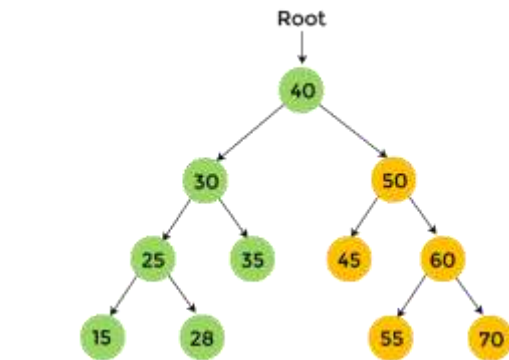
- In left subtree of 25, there is an element 15, and 15 has no subtree. So, **print 15**, and move to the right subtree of 25.



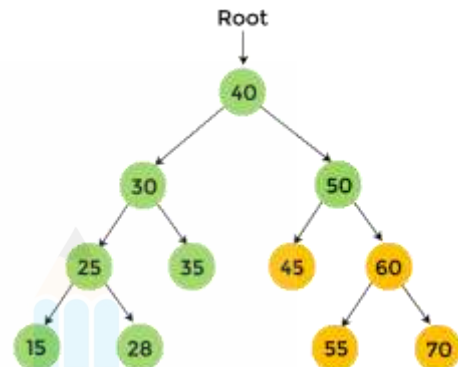
- In right subtree of 25, there is 28, and 28 has no subtree. So, **print 28**, and move to the right subtree of 30.



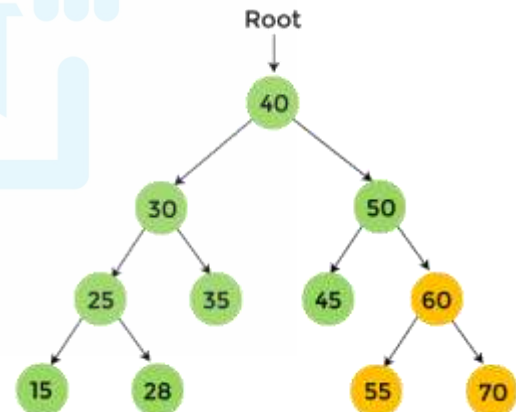
- In right subtree of 30, there is 35 that has no subtree. So **print 35**, and traverse the right subtree of 40.



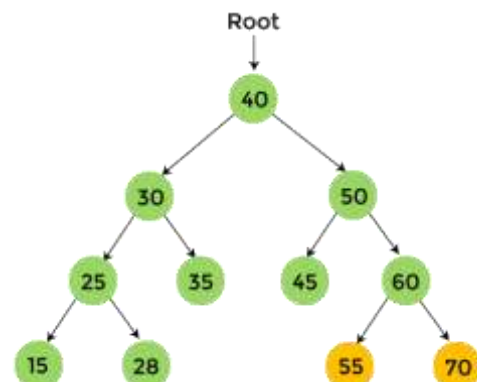
- In the right subtree of 40, there is 50. **Print 50**, and traverse the left subtree of 50.



- In the left subtree of 50, there is 45 that do not have any child. So, **print 45**, and traverse the right subtree of 50.

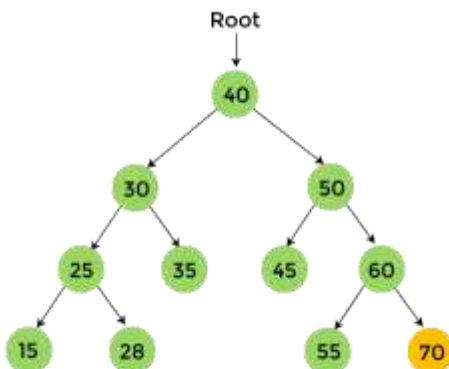


- In right subtree of 50, there is 60. **Print 60** and traverse the left subtree of 60.

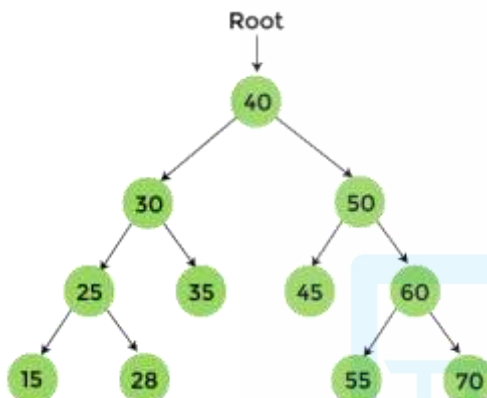




- In the left subtree of 60, there is 55 that does not have any child. So, **print 55** and move to the right subtree of 60.



- In the right subtree of 60, there is 70 that do not have any child. So, **print 70** and stop the process.



After the completion of preorder traversal, the final output is -

40, 30, 25, 15, 28, 35, 50, 45, 60, 55, 70

Complexity of Preorder traversal

The time complexity of preorder traversal is $O(n)$, where 'n' is the size of binary tree.

Whereas, the space complexity of preorder traversal is $O(1)$, if we do not consider the stack size for function calls. Otherwise, the space complexity of preorder traversal is $O(h)$, where 'h' is the height of the tree.

Postorder Traversal

In this article, we will discuss the postorder traversal in data structure.

Linear data structures such as stack, array, queue, etc., only have one way to traverse the data. But in a hierarchical data structure such as **tree**, there are multiple ways to traverse the data. So, here we will discuss another way to traverse the tree data structure,

i.e., **postorder traversal**. The postorder traversal is one of the traversing techniques used for visiting the node in the tree. It follows the principle **LRN (Left-right-node)**. Postorder traversal is used to get the postfix expression of a tree.

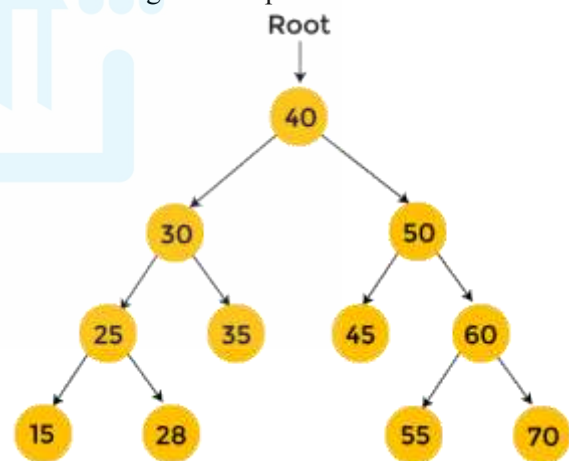
The following steps are used to perform the postorder traversal:

- Traverse the left subtree by calling the postorder function recursively.
- Traverse the right subtree by calling the postorder function recursively.
- Access the data part of the current node.

The post order traversal technique follows the **Left Right Root** policy. Here, Left Right Root means the left subtree of the root node is traversed first, then the right subtree, and finally, the root node is traversed. Here, the Postorder name itself suggests that the tree's root node would be traversed at last.

Example of postorder traversal

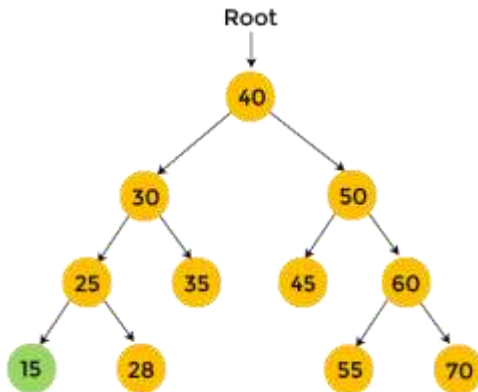
Now, let's see an example of postorder traversal. It will be easier to understand the process of postorder traversal using an example.



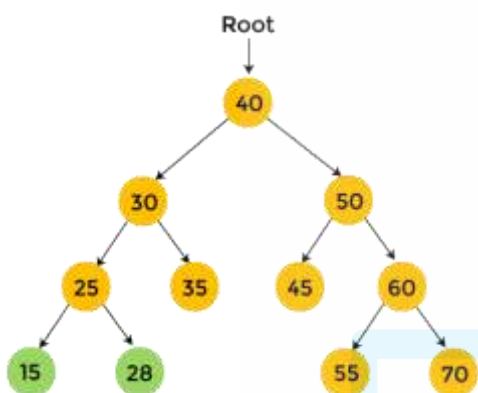
The nodes with yellow color are not visited yet. Now, we will traverse the nodes of above tree using postorder traversal.

- Here, 40 is the root node. We first visit the left subtree of 40, i.e., 30. Node 30 will also traverse in post order. 25 is the left subtree of 30, so it is also traversed in post order. Then 15 is the left subtree of 25. But 15 has no subtree, so **print 15** and move

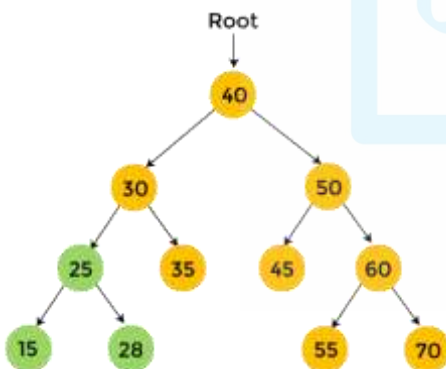
towards the right subtree of 25.



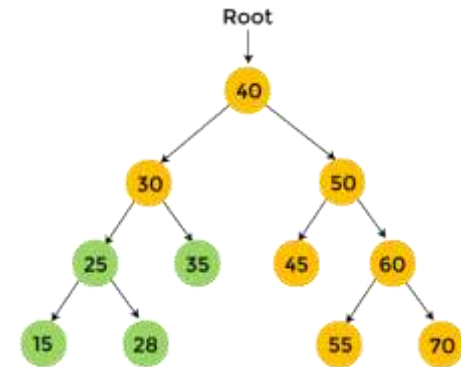
- 28 is the right subtree of 25, and it has no children. So, **print 28**.



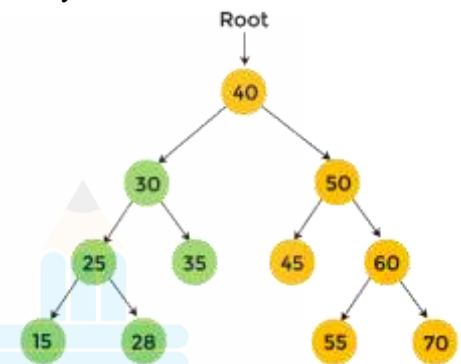
- Now, **print 25**, and the postorder traversal for **25** is finished.



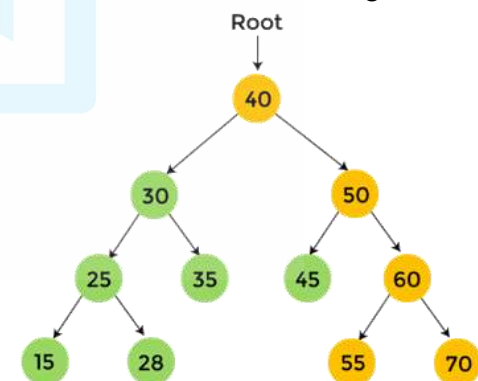
- Next, move towards the right subtree of 30. 35 is the right subtree of 30, and it has no children. So, **print 35**.



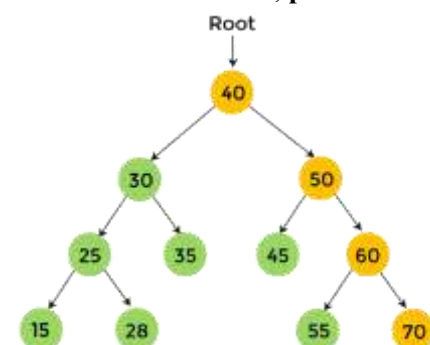
- After that, **print 30**, and the postorder traversal for **30** is finished. So, the left subtree of given binary tree is traversed.



- Now, move towards the right subtree of 40 that is 50, and it will also traverse in post order. 45 is the left subtree of 50, and it has no children. So, **print 45** and move towards the right subtree of 50.

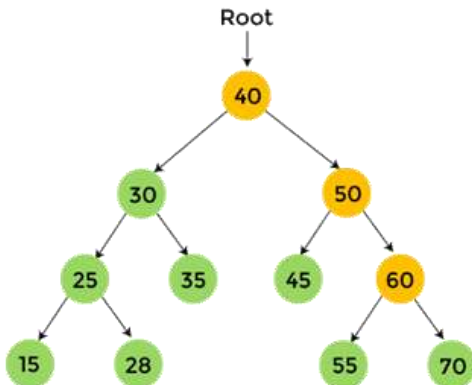


- 60 is the right subtree of 50, which will also be traversed in post order. 55 is the left subtree of 60 that has no children. So, **print 55**.

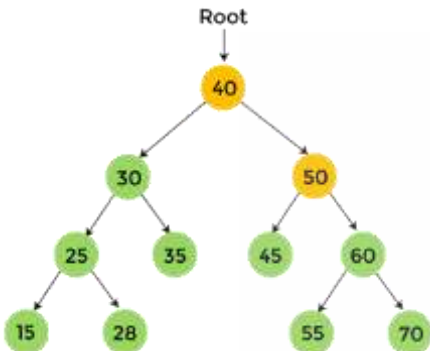




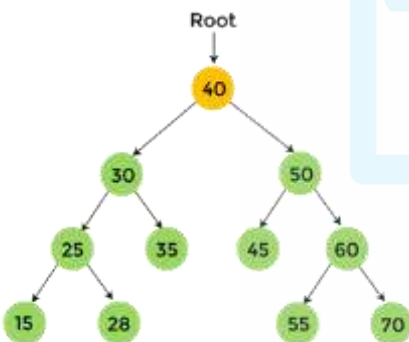
- Now, **print 70**, which is the right subtree of 60.



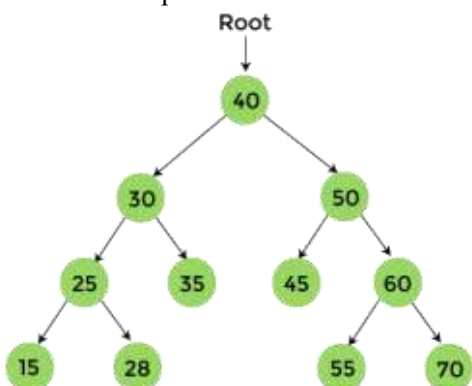
- Now, **print 60**, and the post order traversal for 60 is completed.



- Now, **print 50**, and the post order traversal for 50 is completed.



- At last, **print 40**, which is the root node of the given binary tree, and the post order traversal for node 40 is completed.



The final output that we will get after postorder traversal is -

{15, 28, 25, 35, 30, 45, 55, 70, 60, 50, 40}

Complexity of Postorder traversal

The time complexity of postorder traversal is $O(n)$, where 'n' is the size of binary tree.

Whereas, the space complexity of postorder traversal is $O(1)$, if we do not consider the stack size for function calls. Otherwise, the space complexity of postorder traversal is $O(h)$, where 'h' is the height of tree



Bubble sort Algorithm

- Bubble sort works on the repeatedly swapping of adjacent elements until they are not in the intended order
- Bubbles in water rise up to the surface; similarly, the array elements in bubble sort move to the end in each iteration.
- Although it is simple to use, it is primarily used as an educational tool because the performance of bubble sort is poor in the real world. It is not suitable for large data sets. The average and worst-case complexity of Bubble sort is $O(n^2)$, where n is a number of items.

Bubble sort is majorly used where -

- complexity does not matter
- simple and short code is preferred

Working of Bubble sort Algorithm

Now, let's see the working of Bubble sort Algorithm.

To understand the working of bubble sort algorithm, let's take an unsorted array. We are taking a short and accurate array, as we know the complexity of bubble sort is $O(n^2)$.

Let the elements of array are -

13	32	26	35	10
----	----	----	----	----

First Pass

Sorting will start from the initial two elements. Let compare them to check which is greater.

13	32	26	35	10
----	----	----	----	----

Here, 32 is greater than 13 ($32 > 13$), so it is already sorted. Now, compare 32 with 26.

13	32	26	35	10
----	----	----	----	----

Here, 26 is smaller than 32. So, swapping is required. After swapping new array will look like -

13	26	32	35	10
----	----	----	----	----

Now, compare 32 and 35.

13	26	32	35	10
----	----	----	----	----

Here, 35 is greater than 32. So, there is no swapping required as they are already sorted.

Now, the comparison will be in between 35 and 10.

13	26	32	35	10
----	----	----	----	----

Here, 10 is smaller than 35 that are not sorted. So, swapping is required. Now, we reach at the end of the array. After first pass, the array will be -

13	26	32	10	35
----	----	----	----	----

Now, move to the second iteration.

Second Pass

The same process will be followed for second iteration.

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

Here, 10 is smaller than 32. So, swapping is required. After swapping, the array will be -

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----

Now, move to the third iteration.

Third Pass

The same process will be followed for third iteration.

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----



Here, 10 is smaller than 26. So, swapping is required.

13	10	26	32	35
----	----	----	----	----

13	10	26	32	35
----	----	----	----	----

13	10	26	32	35
----	----	----	----	----

Now, move to the fourth iteration.

Fourth pass

Similarly, after the fourth iteration, the array will be -

10	13	26	32	35
----	----	----	----	----

Hence, there is no swapping required, so the array is completely sorted.

Bubble sort complexity

Now, let's see the time complexity of bubble sort in the best case, average case, and worst case. We will also see the space complexity of bubble sort.

1. Time Complexity

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

- **Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of bubble sort is $O(n)$.
- **Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of bubble sort is $O(n^2)$.
- **Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of bubble sort is $O(n^2)$.

After swapping, the array will be -

2. Space Complexity

Space Complexity	$O(1)$
Stable	YES

- The space complexity of bubble sort is $O(1)$. It is because, in bubble sort, an extra variable is required for swapping.
- The space complexity of optimized bubble sort is $O(2)$. It is because two extra variables are required in optimized bubble sort.

Now, let's discuss the optimized bubble sort algorithm.

Insertion Sort Algorithm

- Insertion sort works similar to the sorting of playing cards in hands. It is assumed that the first card is already sorted in the card game, and then we select an unsorted card. If the selected unsorted card is greater than the first card, it will be placed at the right side; otherwise, it will be placed at the left side. Similarly, all unsorted cards are taken and put in their exact place.
- The same approach is applied in insertion sort. The idea behind the insertion sort is that first take one element, iterate it through the sorted array. Although it is simple to use, it is not appropriate for large data sets as the time complexity of insertion sort in the average case and worst case is $O(n^2)$, where n is the number of items. Insertion sort is less efficient than the other sorting algorithms like heap sort, quick sort, merge sort, etc.

Insertion sort has various advantages such as -

- Simple implementation
- Efficient for small data sets
- Adaptive, i.e., it is appropriate for data sets that are already substantially sorted.

Now, let's see the algorithm of insertion sort.

Working of Insertion sort Algorithm

- Now, let's see the working of the insertion sort Algorithm.
- To understand the working of the insertion sort algorithm, let's take an unsorted array. It will be



easier to understand the insertion sort via an example.

Let the elements of array are -

12	31	25	8	32	17
----	----	----	---	----	----

Initially, the first two elements are compared in insertion sort.

12	31	25	8	32	17
----	----	----	---	----	----

Here, 31 is greater than 12. That means both elements are already in ascending order. So, for now, 12 is stored in a sorted sub-array.

12	31	25	8	32	17
----	----	----	---	----	----

Now, move to the next two elements and compare them.

12	31	25	8	32	17
----	----	----	---	----	----

12	31	25	8	32	17
----	----	----	---	----	----

Here, 25 is smaller than 31. So, 31 is not at correct position. Now, swap 31 with 25. Along with swapping, insertion sort will also check it with all elements in the sorted array.

For now, the sorted array has only one element, i.e. 12. So, 25 is greater than 12. Hence, the sorted array remains sorted after swapping.

12	25	31	8	32	17
----	----	----	---	----	----

Now, two elements in the sorted array are 12 and 25. Move forward to the next elements that are 31 and 8.

12	25	31	8	32	17
----	----	----	---	----	----

12	25	31	8	32	17
----	----	----	---	----	----

Both 31 and 8 are not sorted. So, swap them.

12	25	8	31	32	17
----	----	---	----	----	----

After swapping, elements 25 and 8 are unsorted.

12	25	8	31	32	17
----	----	---	----	----	----

So, swap them.

12	8	25	31	32	17
----	---	----	----	----	----

Now, elements 12 and 8 are unsorted.

12	8	25	31	32	17
----	---	----	----	----	----

So, swap them too.

8	12	25	31	32	17
---	----	----	----	----	----

Now, the sorted array has three items that are 8, 12 and 25. Move to the next items that are 31 and 32.

8	12	25	31	32	17
---	----	----	----	----	----

Hence, they are already sorted. Now, the sorted array includes 8, 12, 25 and 31.

8	12	25	31	32	17
---	----	----	----	----	----

Move to the next elements that are 32 and 17.

8	12	25	31	32	17
---	----	----	----	----	----

17 is smaller than 32. So, swap them.

8	12	25	31	17	32
---	----	----	----	----	----

8	12	25	31	17	32
---	----	----	----	----	----

Swapping makes 31 and 17 unsorted. So, swap them too.

8	12	25	17	31	32
---	----	----	----	----	----

8	12	25	17	31	32
---	----	----	----	----	----

Now, swapping makes 25 and 17 unsorted. So, perform swapping again.

8	12	17	25	31	32
---	----	----	----	----	----

Now, the array is completely sorted.



Insertion sort complexity

Now, let's see the time complexity of insertion sort in best case, average case, and in worst case. We will also see the space complexity of insertion sort.

1. Time Complexity

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

- **Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of insertion sort is $O(n)$.
- **Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of insertion sort is $O(n^2)$.
- **Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of insertion sort is $O(n^2)$.

2. Space Complexity

Space Complexity	$O(1)$
Stable	YES

- The space complexity of insertion sort is $O(1)$. It is because, in insertion sort, an extra variable is required for swapping.

Quick Sort Algorithm

Sorting is a way of arranging items in a systematic manner. Quicksort is the widely used sorting algorithm that makes $n \log n$ comparisons in average case for sorting an array of n elements.

- It is a faster and highly efficient sorting algorithm.
- This algorithm follows the divide and conquer approach.
- Divide and conquer is a technique of breaking down the algorithms into subproblems, then solving the subproblems, and combining the results back together to solve the original problem.

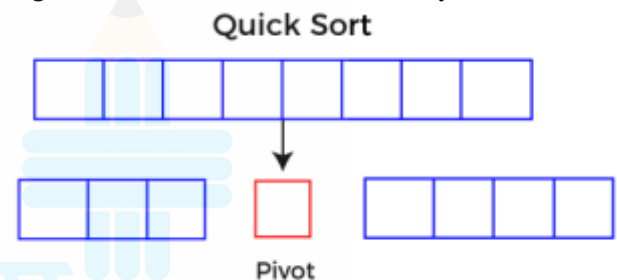
Divide: In Divide, first pick a pivot element. After that, partition or rearrange the array into two sub-arrays such that each element in the left sub-array is less than or equal to the pivot element and each element in the right sub-array is larger than the pivot element.

Conquer: Recursively, sort two subarrays with Quicksort.

Combine: Combine the already sorted array.

Quicksort picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.

After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.



Choosing the pivot

Picking a good pivot is necessary for the fast implementation of quicksort. However, it is typical to determine a good pivot. Some of the ways of choosing a pivot are as follows -

- Pivot can be random, i.e. select the random pivot from the given array.
- Pivot can either be the rightmost element of the leftmost element of the given array.
- Select median as the pivot element.

Working of Quick Sort Algorithm

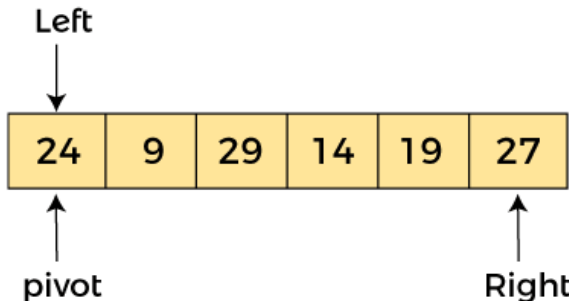
Now, let's see the working of the Quicksort Algorithm. To understand the working of quick sort, let's take an unsorted array. It will make the concept more clear and understandable.

Let the elements of array are -

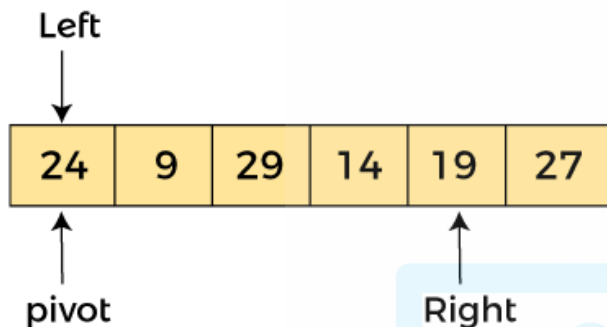
24	9	29	14	19	27
----	---	----	----	----	----

In the given array, we consider the leftmost element as pivot. So, in this case, $a[\text{left}] = 24$, $a[\text{right}] = 27$ and $a[\text{pivot}] = 24$.

Since, pivot is at left, so algorithm starts from right and move towards left.

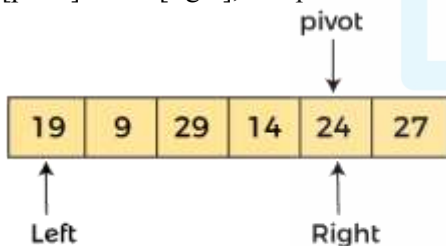


Now, $a[\text{pivot}] < a[\text{right}]$, so algorithm moves forward one position towards left, i.e. -



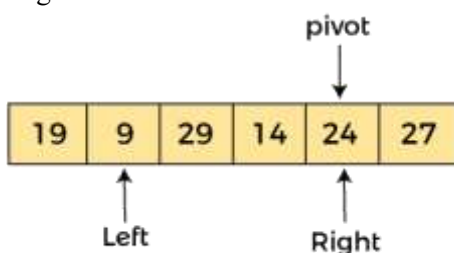
Now, $a[\text{left}] = 24$, $a[\text{right}] = 19$, and $a[\text{pivot}] = 24$.

Because, $a[\text{pivot}] > a[\text{right}]$, so, algorithm will swap $a[\text{pivot}]$ with $a[\text{right}]$, and pivot moves to right, as -

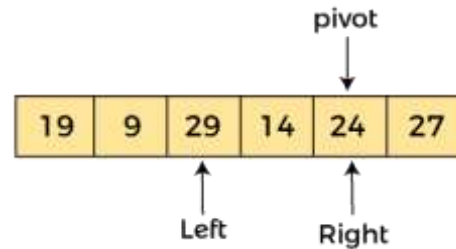


Now, $a[\text{left}] = 19$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. Since, pivot is at right, so algorithm starts from left and moves to right.

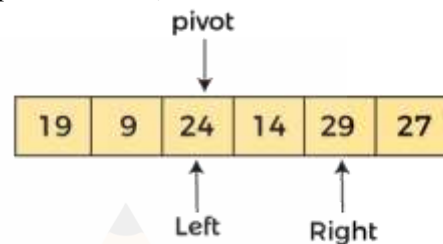
As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as -



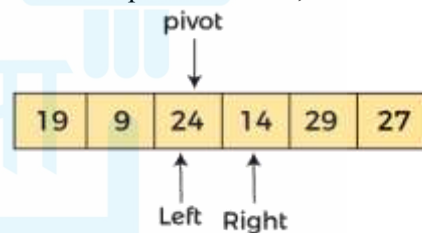
Now, $a[\text{left}] = 9$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as -



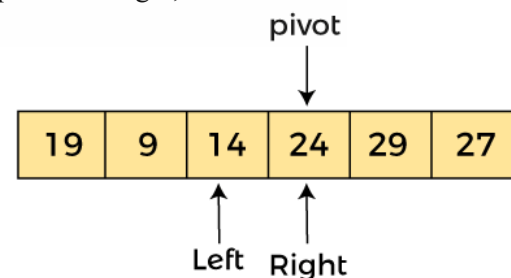
Now, $a[\text{left}] = 29$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{left}]$, so, swap $a[\text{pivot}]$ and $a[\text{left}]$, now pivot is at left, i.e. -



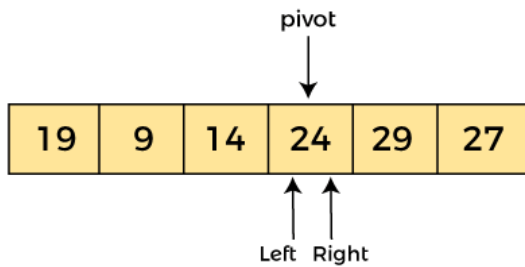
Since, pivot is at left, so algorithm starts from right, and move to left. Now, $a[\text{left}] = 24$, $a[\text{right}] = 29$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{right}]$, so algorithm moves one position to left, as -



Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 24$, and $a[\text{right}] = 14$. As $a[\text{pivot}] > a[\text{right}]$, so, swap $a[\text{pivot}]$ and $a[\text{right}]$, now pivot is at right, i.e. -



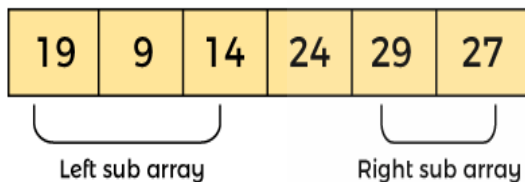
Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 14$, and $a[\text{right}] = 24$. Pivot is at right, so the algorithm starts from left and move to right.



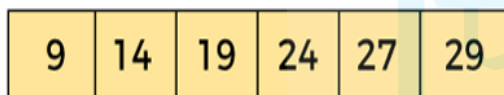
Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 24$, and $a[\text{right}] = 24$. So, pivot, left and right are pointing the same element. It represents the termination of procedure.

Element 24, which is the pivot element is placed at its exact position.

Elements that are right side of element 24 are greater than it, and the elements that are left side of element 24 are smaller than it.



Now, in a similar manner, quick sort algorithm is separately applied to the left and right sub-arrays. After sorting gets done, the array will be -



Quicksort complexity

Now, let's see the time complexity of quicksort in best case, average case, and in worst case. We will also see the space complexity of quicksort.

1. Time Complexity

Case	Time Complexity
Best Case	$O(n \cdot \log n)$
Average Case	$O(n \cdot \log n)$
Worst Case	$O(n^2)$

- **Best Case Complexity** - In Quicksort, the best-case occurs when the pivot element is the middle element or near to the middle element. The best-case time complexity of quicksort is $O(n \cdot \log n)$.
- **Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending.

The average case time complexity of quicksort is $O(n \cdot \log n)$.

- **Worst Case Complexity** - In quick sort, worst case occurs when the pivot element is either greatest or smallest element. Suppose, if the pivot element is always the last element of the array, the worst case would occur when the given array is sorted already in ascending or descending order. The worst-case time complexity of quicksort is $O(n^2)$.

Though the worst-case complexity of quicksort is more than other sorting algorithms such as **Merge sort** and **Heap sort**, still it is faster in practice. Worst case in quick sort rarely occurs because by changing the choice of pivot, it can be implemented in different ways. Worst case in quicksort can be avoided by choosing the right pivot element.

2. Space Complexity

Space Complexity	$O(n \cdot \log n)$
Stable	NO

- The space complexity of quicksort is $O(n \cdot \log n)$.

Selection Sort Algorithm

- In selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted to its appropriate position into the array.
- It is also the simplest algorithm.
- It is an in-place comparison sorting algorithm. In this algorithm, the array is divided into two parts, first is sorted part, and another one is the unsorted part.
- Initially, the sorted part of the array is empty, and unsorted part is the given array. Sorted part is placed at the left, while the unsorted part is placed at the right.
- In selection sort, the first smallest element is selected from the unsorted array and placed at the first position.
- After that second smallest element is selected and placed in the second position. The process continues until the array is entirely sorted.



- The average and worst-case complexity of selection sort is $O(n^2)$, where n is the number of items. Due to this, it is not suitable for large data sets.

Selection sort is generally used when -

- A small array is to be sorted
- Swapping cost doesn't matter
- It is compulsory to check all elements

Working of Selection sort Algorithm

Now, let's see the working of the Selection sort Algorithm.

To understand the working of the Selection sort algorithm, let's take an unsorted array. It will be easier to understand the Selection sort via an example.

Let the elements of array are -

12 29 25 8 32 17 40

Now, for the first position in the sorted array, the entire array is to be scanned sequentially.

At present, 12 is stored at the first position, after searching the entire array, it is found that 8 is the smallest value.

12 29 25 8 32 17 40

So, swap 12 with 8. After the first iteration, 8 will appear at the first position in the sorted array.

8 29 25 12 32 17 40

For the second position, where 29 is stored presently, we again sequentially scan the rest of the items of unsorted array. After scanning, we find that 12 is the second lowest element in the array that should be appeared at second position.

8 29 25 12 32 17 40

Now, swap 29 with 12. After the second iteration, 12 will appear at the second position in the sorted array. So, after two iterations, the two smallest values are placed at the beginning in a sorted way.

8 12 25 29 32 17 40

The same process is applied to the rest of the array elements. Now, we are showing a pictorial representation of the entire sorting process.

8 12 25 29 32 17 40

8 12 25 29 32 17 40

8 12 17 29 32 25 40

8 12 17 29 32 25 40

8 12 17 29 32 25 40

8 12 17 25 32 29 40

8 12 17 25 32 29 40

8 12 17 25 32 29 40

8 12 17 25 29 32 40

8 12 17 25 29 32 40

Now, the array is completely sorted.

Selection sort complexity

Now, let's see the time complexity of selection sort in best case, average case, and in worst case. We will also see the space complexity of the selection sort.

1. Time Complexity

Case	Time Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

- Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of selection sort is $O(n^2)$.
- Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of selection sort is $O(n^2)$.



- **Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of selection sort is $O(n^2)$.

2. Space Complexity

Space Complexity	$O(1)$
Stable	YES

- The space complexity of selection sort is $O(1)$. It is because, in selection sort, an extra variable is required for swapping.

Merge Sort Algorithm

- Merge sort is the sorting technique that follows the divide and conquer approach.
- This article will be very helpful and interesting to students as they might face merge sort as a question in their examinations
- Merge sort is similar to the quick sort algorithm as it uses the divide and conquer approach to sort the elements.
- It is one of the most popular and efficient sorting algorithm.
- It divides the given list into two equal halves, calls itself for the two halves and then merges the two sorted halves.
- We have to define the **merge()** function to perform the merging.
- The sub-lists are divided again and again into halves until the list cannot be divided further.
- Then we combine the pair of one element lists into two-element lists, sorting them in the process. The sorted two-element pairs is merged into the four-element lists, and so on until we get the sorted list.
- Now, let's see the algorithm of merge sort.

Working of Merge sort Algorithm

Now, let's see the working of merge sort Algorithm.

To understand the working of the merge sort algorithm, let's take an unsorted array. It will be easier to understand the merge sort via an example.

Let the elements of array are -

12	31	25	8	32	17	40	42
----	----	----	---	----	----	----	----

According to the merge sort, first divide the given array into two equal halves. Merge sort keeps dividing the list into equal parts until it cannot be further divided.

As there are eight elements in the given array, so it is divided into two arrays of size 4.

divide	12	31	25	8	32	17	40	42
--------	----	----	----	---	----	----	----	----

Now, again divide these two arrays into halves. As they are of size 4, so divide them into new arrays of size 2.

divide	12	31	25	8	32	17	40	42
--------	----	----	----	---	----	----	----	----

Now, again divide these arrays to get the atomic value that cannot be further divided.

divide	12	31	25	8	32	17	40	42
--------	----	----	----	---	----	----	----	----

- Now, combine them in the same manner they were broken.
- In combining, first compare the element of each array and then combine them into another array in sorted order.
- So, first compare 12 and 31, both are in sorted positions. Then compare 25 and 8, and in the list of two values, put 8 first followed by 25. Then compare 32 and 17, sort them and put 17 first followed by 32. After that, compare 40 and 42, and place them sequentially.

merge	12	31	8	25	17	32	40	42
-------	----	----	---	----	----	----	----	----

In the next iteration of combining, now compare the arrays with two data values and merge them into an array of four values in sorted order.



merge

8	12	25	31	17	32	40	42
---	----	----	----	----	----	----	----

Now, there is a final merging of the arrays. After the final merging of above arrays, the array will look like -

8	12	17	25	31	32	40	42
---	----	----	----	----	----	----	----

Now, the array is completely sorted.

Merge sort complexity

Now, let's see the time complexity of merge sort in best case, average case, and in worst case. We will also see the space complexity of the merge sort.

1. Time Complexity

Case	Time Complexity
Best Case	$O(n \cdot \log n)$
Average Case	$O(n \cdot \log n)$
Worst Case	$O(n \cdot \log n)$

- **Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of merge sort is $O(n \cdot \log n)$.
- **Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of merge sort is $O(n \cdot \log n)$.
- **Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of merge sort is $O(n \cdot \log n)$.

2. Space Complexity

Space Complexity	$O(n)$
Stable	YES

- The space complexity of merge sort is $O(n)$. It is because, in merge sort, an extra variable is required for swapping.

Heap Sort Algorithm

Heap sort processes the elements by creating the min-heap or max-heap using the elements of the given array. Min-heap or max-heap represents the ordering of array in which the root element represents the minimum or maximum element of the array.

Heap sort basically recursively performs two main operations -

- Build a heap H , using the elements of array.
- Repeatedly delete the root element of the heap formed in 1st phase.

Before knowing more about the heap sort, let's first see a brief description of **Heap**.

What is a heap?

A heap is a complete binary tree, and the binary tree is a tree in which the node can have the utmost two children. A complete binary tree is a binary tree in which all the levels except the last level, i.e., leaf node, should be completely filled, and all the nodes should be left-justified.

What is heap sort?

Heapsort is a popular and efficient sorting algorithm. The concept of heap sort is to eliminate the elements one by one from the heap part of the list, and then insert them into the sorted part of the list.

Working of Heap sort Algorithm

Now, let's see the working of the Heapsort Algorithm.

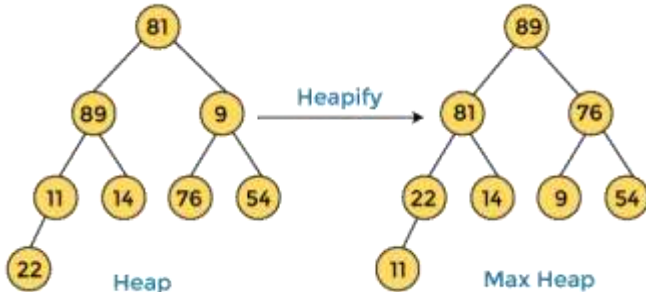
In heap sort, basically, there are two phases involved in the sorting of elements. By using the heap sort algorithm, they are as follows -

- The first step includes the creation of a heap by adjusting the elements of the array.
- After the creation of heap, now remove the root element of the heap repeatedly by shifting it to the end of the array, and then store the heap structure with the remaining elements.

Now let's see the working of heap sort in detail by using an example. To understand it more clearly, let's take an unsorted array and try to sort it using heap sort. It will make the explanation clearer and easier.

81	89	9	11	14	76	54	22
----	----	---	----	----	----	----	----

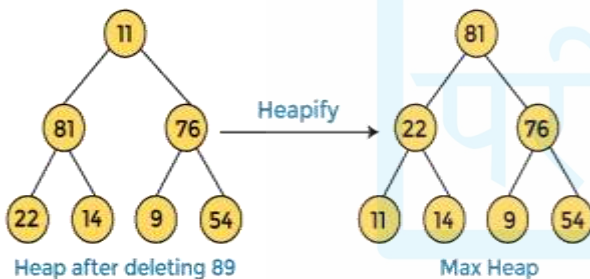
First, we have to construct a heap from the given array and convert it into max heap.



After converting the given heap into max heap, the array elements are -

89	81	76	22	14	9	54	11
----	----	----	----	----	---	----	----

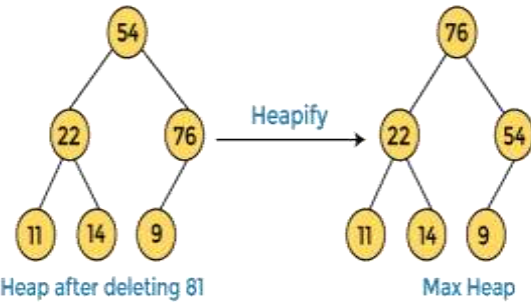
Next, we have to delete the root element (89) from the max heap. To delete this node, we have to swap it with the last node, i.e. (11). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 89 with 11, and converting the heap into max-heap, the elements of array are -

81	22	76	11	14	9	54	89
----	----	----	----	----	---	----	----

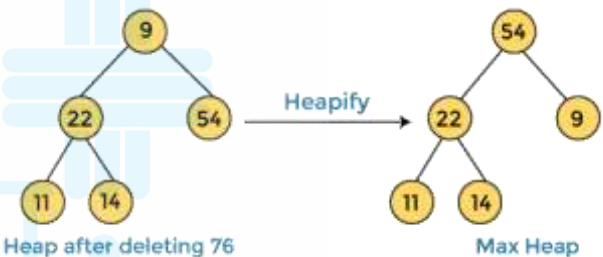
In the next step, again, we have to delete the root element (81) from the max heap. To delete this node, we have to swap it with the last node, i.e. (54). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 81 with 54 and converting the heap into max-heap, the elements of array are -

76	22	54	11	14	9	81	89
----	----	----	----	----	---	----	----

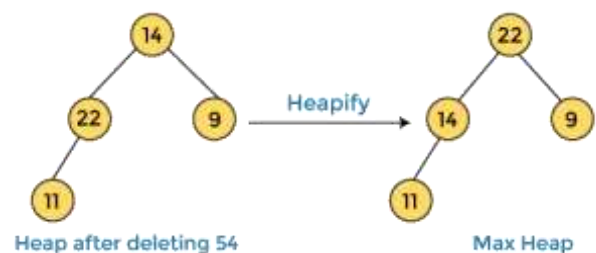
In the next step, we have to delete the root element (76) from the max heap again. To delete this node, we have to swap it with the last node, i.e. (9). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 76 with 9 and converting the heap into max-heap, the elements of array are -

54	22	9	11	14	76	81	89
----	----	---	----	----	----	----	----

In the next step, again we have to delete the root element (54) from the max heap. To delete this node, we have to swap it with the last node, i.e. (14). After deleting the root element, we again have to heapify it to convert it into max heap.

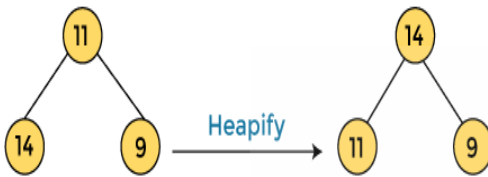




After swapping the array element **54** with **14** and converting the heap into max-heap, the elements of array are -

22	14	9	11	54	76	81	89
----	----	---	----	----	----	----	----

In the next step, again we have to delete the root element (**22**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**11**). After deleting the root element, we again have to heapify it to convert it into max heap.



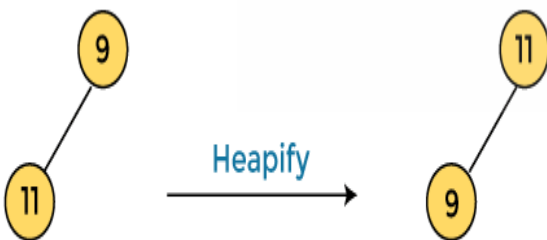
Heap after deleting 22

Max Heap

After swapping the array element **22** with **11** and converting the heap into max-heap, the elements of array are -

14	11	9	22	54	76	81	89
----	----	---	----	----	----	----	----

In the next step, again we have to delete the root element (**14**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**9**). After deleting the root element, we again have to heapify it to convert it into max heap.



Heap after deleting 14

Max Heap

After swapping the array element **14** with **9** and converting the heap into max-heap, the elements of array are -

11	9	14	22	54	76	81	89
----	---	----	----	----	----	----	----

In the next step, again we have to delete the root element (**11**) from the max heap. To delete this node,

we have to swap it with the last node, i.e. (**9**). After deleting the root element, we again have to heapify it to convert it into max heap.



Heap after deleting 11

Max Heap

After swapping the array element **11** with **9**, the elements of array are -

9	11	14	22	54	76	81	89
---	----	----	----	----	----	----	----

Now, heap has only one element left. After deleting it, heap will be empty.



After completion of sorting, the array elements are -

9	11	14	22	54	76	81	89
---	----	----	----	----	----	----	----

Now, the array is completely sorted.

Heap sort complexity

Now, let's see the time complexity of Heap sort in the best case, average case, and worst case. We will also see the space complexity of Heapsort.

1. Time Complexity

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

- **Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of heap sort is **$O(n \log n)$** .
- **Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of heap sort is **$O(n \log n)$** .



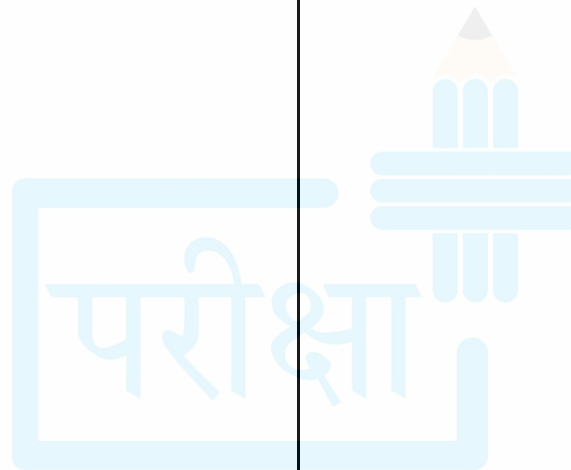
- **Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of heap sort is **$O(n \log n)$** .

The time complexity of heap sort is **$O(n \log n)$** in all three cases (best case, average case, and worst case). The height of a complete binary tree having n elements is **$\log n$** .

2. Space Complexity

Space Complexity	$O(1)$
Stable	No

- The space complexity of Heap sort is $O(1)$.

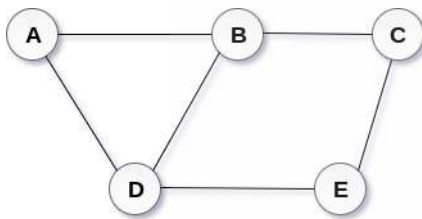


Graph

A graph can be defined as group of vertices and edges that are used to connect these vertices.

Definition

- A graph G can be defined as an ordered set $G(V, E)$ where $V(G)$ represents the set of vertices and $E(G)$ represents the set of edges which are used to connect these vertices.
- A Graph $G(V, E)$ with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A)) is shown in the following figure.

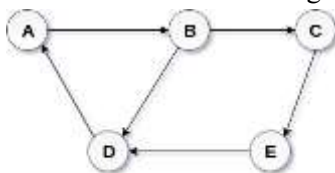


Undirected Graph

Directed and Undirected Graph

- A graph can be directed or undirected.
- However, in an undirected graph, edges are not associated with the directions with them.
- An undirected graph is shown in the above figure since its edges are not attached with any of the directions.
- If an edge exists between vertex A and B then the vertices can be traversed from B to A as well as A to B.
- **In a directed graph**, edges form an ordered pair. Edges represent a specific path from some vertex A to another vertex B. Node A is called initial node while node B is called terminal node.

A directed graph is shown in the following figure.



Directed Graph

Graph Terminology

Path

A path can be defined as the sequence of nodes that are followed in order to reach some terminal node V from the initial node U .

Closed Path

A path will be called as closed path if the initial node is same as terminal node. A path will be closed path if $V_0 = V_N$.

Simple Path

If all the nodes of the graph are distinct with an exception $V_0 = V_N$, then such path P is called as closed simple path.

Cycle

A cycle can be defined as the path which has no repeated edges or vertices except the first and last vertices.

Connected Graph

A connected graph is the one in which some path exists between every two vertices (u, v) in V . There are no isolated nodes in connected graph.

Complete Graph

A complete graph is the one in which every node is connected with all other nodes. A complete graph contain $n(n-1)/2$ edges where n is the number of nodes in the graph.

Weighted Graph

In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge e can be given as $w(e)$ which must be a positive (+) value indicating the cost of traversing the edge.

Loop

An edge that is associated with the similar end points can be called as Loop.

Adjacent Nodes

If two nodes u and v are connected via an edge e , then the nodes u and v are called as neighbours or adjacent nodes.

Degree of the Node

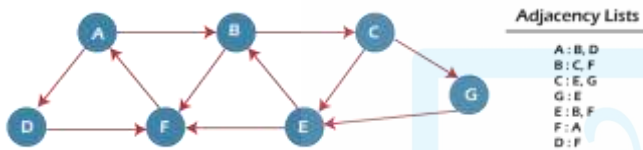
A degree of a node is the number of edges that are connected with that node. A node with degree 0 is called as isolated node.

BFS algorithm

- Breadth-first search is a graph traversal algorithm that starts traversing the graph from the root node and explores all the neighboring nodes. Then, it selects the nearest node and explores all the unexplored nodes. While using BFS for traversal, any node in the graph can be considered as the root node.
- It is a recursive algorithm to search all the vertices of a tree or graph data structure. BFS puts every vertex of the graph into two categories - visited and non-visited. It selects a single node in a graph and, after that, visits all the nodes adjacent to the selected node.

Example of BFS algorithm

Now, let's understand the working of BFS algorithm by using an example. In the example given below, there is a directed graph having 7 vertices.



In the above graph, minimum path 'P' can be found by using the BFS that will start from Node A and end at Node E. The algorithm uses two queues, namely QUEUE1 and QUEUE2. QUEUE1 holds all the nodes that are to be processed, while QUEUE2 holds all the nodes that are processed and deleted from QUEUE1. Now, let's start examining the graph starting from Node A.

Step 1 - First, add A to queue1 and NULL to queue2.

QUEUE1 = {A}

QUEUE2 = {NULL}

Step 2 - Now, delete node A from queue1 and add it into queue2. Insert all neighbors of node A to queue1.

QUEUE1 = {B, D}

QUEUE2 = {A}

Step 3 - Now, delete node B from queue1 and add it into queue2. Insert all neighbors of node B to queue1.

QUEUE1 = {D, C, F}

QUEUE2 = {A, B}

Step 4 - Now, delete node D from queue1 and add it into queue2. Insert all neighbors of node D to queue1.

The only neighbor of Node D is F since it is already inserted, so it will not be inserted again.

QUEUE1 = {C, F}

QUEUE2 = {A, B, D}

Step 5 - Delete node C from queue1 and add it into queue2. Insert all neighbors of node C to queue1.

QUEUE1 = {F, E, G}

QUEUE2 = {A, B, D, C}

Step 6 - Delete node F from queue1 and add it into queue2. Insert all neighbors of node F to queue1. Since all the neighbors of node F are already present, we will not insert them again.

QUEUE1 = {E, G}

QUEUE2 = {A, B, D, C, F}

Step 7 - Delete node E from queue1. Since all of its neighbors have already been added, so we will not insert them again. Now, all the nodes are visited, and the target node E is encountered into queue2.

QUEUE1 = {G}

QUEUE2 = {A, B, D, C, F, E}

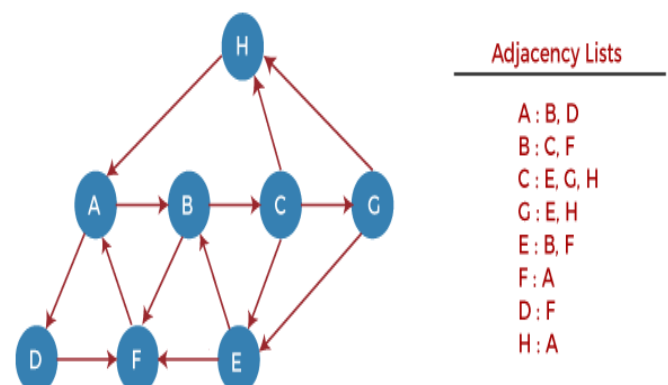
DFS (Depth First Search) algorithm

It is a recursive algorithm to search all the vertices of a tree data structure or a graph. The depth-first search (DFS) algorithm starts with the initial node of graph G and goes deeper until we find the goal node or the node with no children.

Because of the recursive nature, stack data structure can be used to implement the DFS algorithm.

Example of DFS algorithm

Now, let's understand the working of the DFS algorithm by using an example. In the example given below, there is a directed graph having 7 vertices.





Now, let's start examining the graph starting from Node H.

Step 1 - First, push H onto the stack.

STACK: H

Step 2 - POP the top element from the stack, i.e., H, and print it. Now, PUSH all the neighbors of H onto the stack that are in ready state.

Print: H]STACK: A

Step 3 - POP the top element from the stack, i.e., A, and print it. Now, PUSH all the neighbors of A onto the stack that are in ready state.

Print: A

STACK: B, D

Step 4 - POP the top element from the stack, i.e., D, and print it. Now, PUSH all the neighbors of D onto the stack that are in ready state.

Print: D

STACK: B, F

Step 5 - POP the top element from the stack, i.e., F, and print it. Now, PUSH all the neighbors of F onto the stack that are in ready state.

Print: F

STACK: B

Step 6 - POP the top element from the stack, i.e., B, and print it. Now, PUSH all the neighbors of B onto the stack that are in ready state.

Print: B

STACK: C

Step 7 - POP the top element from the stack, i.e., C, and print it. Now, PUSH all the neighbors of C onto the stack that are in ready state.

Print: C

STACK: E, G

Step 8 - POP the top element from the stack, i.e., G and PUSH all the neighbors of G onto the stack that are in ready state.

Print: G

STACK: E

Step 9 - POP the top element from the stack, i.e., E and PUSH all the neighbors of E onto the stack that are in ready state.

Print: E

STACK:



MCQ

Q1. What would be the output after performing the following operations in a Deque?

1. Insertfront(10);
2. Insertfront(20);
3. Insertrear(30);
4. Insertrear(40);
5. Deletefront();
6. Insertfront(50);
7. Deleterear();
8. Display();

- (A) 10, 20, 30
 (B) 50, 10, 30
 (C) 40, 20, 30
 (D) None of the above

Answer: B

Explanation:

The answer is b.

Let's dry run the above code.

When insertfront(10) is called, deque would be:

10

When insertfront(20) is called, the deque would be:

20 10

When insertrear(30) is called, the deque would be:

20 10 30

When insertrear(40) is called, the deque would be:

20 10 30 40

When deletefront() is called, the deque would be:

10 30 40

When insertfront(50) is called, the deque would be:

50 10 30 40

When deleterear() is called, the deque would be:

50 10 30

Q2. In Queue, Inserting elements beyond capacity of the queue will result in an exception known as _____.

- (A) Underflow
 (B) Overflow
 (C) Underflow and Overflow
 (D) More than one of the above
 (E) None of the above

Answer: B

Concept:

Queues:

- The queue is an abstract data structure, somewhat similar to Stacks.
- Unlike stacks, a queue is open at both ends. One end is always used to insert data (en queue) and the other is used to remove data (dequeue). Queue follows the First-In-First-Out methodology, i.e., the data item stored first will be accessed first.
- Queue follows the principle of First In First Out (FIFO) and Last In Last Out (LILO).

ENQUEUE:

It is used to insert a new element into the queue at the rear end. We can insert elements in the queue till there is space in the queue for adding more elements. Inserting elements beyond the capacity of the queue will result in an exception - known as **Overflow**.

Hence the correct answer is Overflow.

Q3. Which of the following data structure is follows for train ticket is in the waiting list?

- (A) stack
 (B) queue
 (C) tree
 (D) More than one of the above
 (E) None of the above

Answer: B

Concept:

Queues:

- The queue is an abstract data structure, somewhat similar to Stacks.
- Unlike stacks, a queue is open at both ends. One end is always used to insert data (en queue) and the other is used to remove data (dequeue). Queue follows the First-In-First-Out methodology, i.e., the data item stored first will be accessed first.
- Queue follows the principle of First In First Out (FIFO) and Last In Last Out (LILO)

Explanation:

If a train ticket is on the waiting list (such as W/L1), it means the ticket is in a queue of tickets waiting to get confirmed, as per the increasing order of waiting numbers. If a confirmed ticket is canceled, the W/ L1



numbered ticket is removed from the FRONT of the waiting queue and confirmed.

It follows the queue data structure. Stack is not suitable because it follows the first in last out (FILO) manner. Tree and linked lists are also not suitable because storing the previous node is required and they are not aware of which node came first.

Hence the correct answer is the queue.

Q4. Which of the following strategy is used for queue data structure?

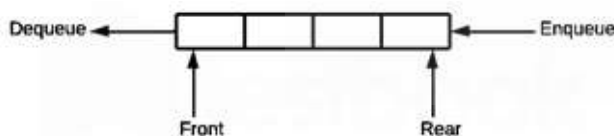
- (A) First in first out
- (B) First in last out
- (C) Last in first out
- (D) More than one of the above
- (E) None of the above

Answer: A

Concept:

Queues:

- The queue is an abstract data structure, somewhat similar to Stacks.
- Unlike stacks, a queue is open at both ends. One end is always used to insert data (en queue) and the other is used to remove data (dequeue). Queue follows the First-In-First-Out methodology, i.e., the data item stored first will be accessed first.
- Queue follows the principle of First In First Out (FIFO) and Last In Last Out (LILO)



Hence the correct answer is First in first out.

Q5. In queue, Insertion is happens at the _____ end.

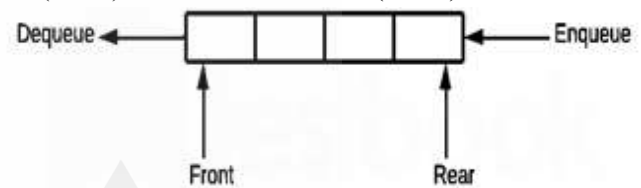
- (A) Rear
- (B) front
- (C) either rear or front
- (D) More than one of the above
- (E) None of the above

Answer: A

Concept:

Queue:

- The queue is an abstract data structure, somewhat similar to Stacks.
- Insertion in a queue happens at the **rear end** and deletion happens at the **front end**.
- Unlike stacks, a queue is open at both ends. One end is always used to insert data (en queue) and the other is used to remove data (dequeue). Queue follows the First-In-First-Out methodology, i.e., the data item stored first will be accessed first.
- Queue follows the principle of First In First Out (FIFO) and Last In Last Out (LILO)



Hence the correct answer is Rear.

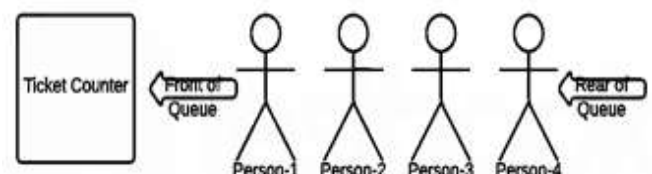
Q6. He LILO and FIFO methods are followed by which data structure?

- (A) Stack
- (B) Queue
- (C) Array
- (D) More than one of the above
- (E) None of the above

Answer: B

The correct answer is Queue.

- The queue is the data structure where the insertion and deletion take place from different ends. The insertion takes place from the rear end while the deletion takes place from the front end.
- For instance: The queue of people for a movie ticket. The person who is added first to the queue will get the ticket first and also get removed first. The person who joined the queue, last, will get ticked at last and also get removed last from the queue.





- The person-1 is added first to the queue and also will get deleted first from the queue.
- Similarly, the element inserted first in the queue also gets removed first from the queue.
- The stack follows LIFO (Last In First Out) and FILO (First In Last Out) methods to insert and delete the elements.
- The array does not follow any method to insert or delete the elements, the array can be accessed randomly.

Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions.

Q7. Which one of the following statements is TRUE with respect to this modified stack?

- (A) A queue cannot be implemented using this stack.
- (B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions.
- (C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.
- (D) A queue can be implemented where both ENQUEUE and DEQUEUE take a single instruction each.

Answer: C

Option 3 : A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.

Explanation:

We want to implement queue using stack which has three functions reverse, push and pop.

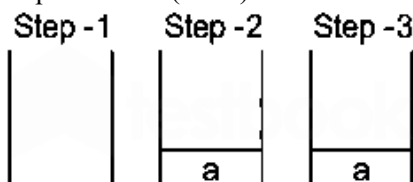
Example:

Suppose we want to insert a,b in queue using stack then ENQUEUE item a:

Step 1: reverse (stack)

Step 2: push (a)

Step 3: reverse(stack)

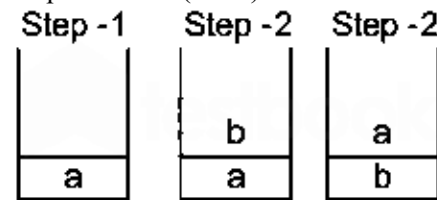


ENQUEUE item b:

Step 1: reverse (stack)

Step 2: push (b)

Step 3: reverse(stack)



- After inserting b to stack if we want to DEQUEUE then only pop of top element is sufficient.
- So for ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.
- Similarly DEQUEUE (reverse pop reverse) and ENQUEUE (push) is also valid for queue implementation.

Q8. In which of the following data structures, both insertion and deletion operations can take place at either end of the structure?

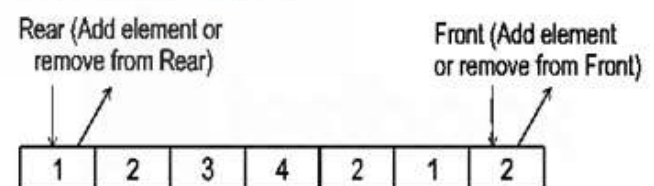
- (A) Deque (B) Priority queue
- (C) Stack (D) Queue

Answer: A

Concept: Deque is also known as double ended queue. New items can be added at either the front or the rear. Likewise existing item can be removed from either end.

Explanation: (Diagram)

Explanation: (Diagram)



Hence option 1 is the correct answer.

Q9. Typical time requirement for operations on queues is:

- (A) $O(1)$
- (B) $O(n)$
- (C) $O(\log n)$
- (D) $O(n^2)$

Answer: A



Typical time requirement for operations on queues is $O(1)$

Insertion

Insertion operation in a queue, a new node is added, new node points to rear pointer of the queue and the rear pointer points to new node; This operation takes a constant amount of time, Therefore, the complexity of insertion in the queue is $O(1)$.

Deletion

Deletion operation in a queue, element to be deleted is a front element, address of the front pointer is stored in a temporary variable, the front pointer points to its immediate next node and location stored in a temporary variable is free; This operation takes a constant amount of time, Therefore, the complexity of insertion in a queue is $O(1)$

- Queue is data structure which is First in First Out (FIFO) or Last in Last Out (LILO) which means the element which is inserted first comes out first from the queue or the elements inserted last comes out last from the queue
- Breadth First Search (BFS) algorithm traverses a graph in a breadthwise manner and uses a queue to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

Q10. The minimum number of stacks needed to implement a queue is

- (A) 3
(B) 1
(C) 2
(D) 4

Answer: C

Keep two stacks P and Q

Enqueue:

Push the new element onto P

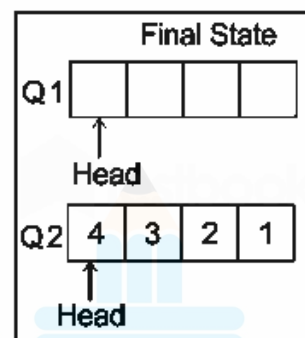
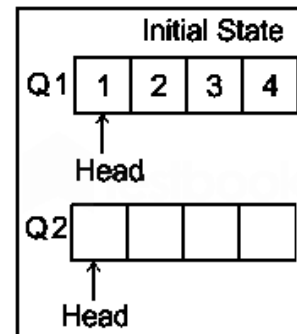
Dequeue:

If Q is empty, refill it by popping each element from P and pushing it onto Q

Pop and return the top element from Q.

Consider the queues Q_1 containing four elements and Q_2 containing none (shown as the Initial State in the figure). The only operations allowed on these two queues are Enqueue(Q, element) and Dequeue(Q). The

minimum number of Enqueue operations on Q_1 required to place the elements of Q_1 in Q_2 in reverse order (shown as the Final State in the figure) without using any additional storage is _____.



The correct answer is 0.

Concept:

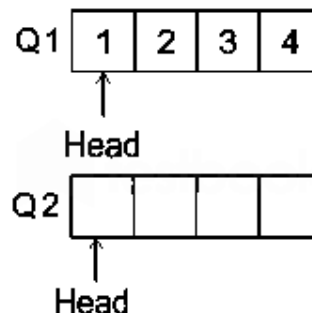
A queue is a linear structure in which operations are carried out in a specific order. First in, first out is the order (FIFO). It has two operations enqueue and dequeue. Enqueue is used to insert the element and dequeue is used to delete the element from the queue.

Explanation:

Initially, they haven't mentioned the size of the queue and type of queue (circular queue or normal queue).

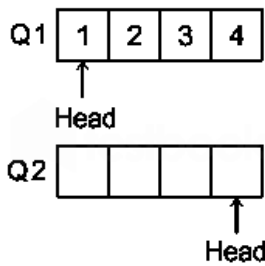
So consider that Q_1 and Q_2 are normal queues and there is no restriction on size.

Step1: Given queues Q_1 and Q_2 . These queues are array manner queues look like,



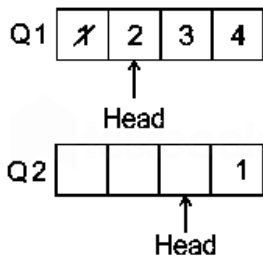


Step 2: Traverse head of Q2 at the end of queue so Queue format becomes.



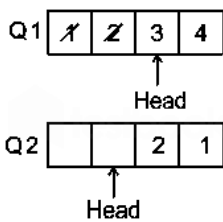
Step 3: Now perform the dequeue operation in Q1 at the head position and perform enqueue operation in Q2 at the head position.

- Dequeue(Q1)
- Enqueue(Q2,1)



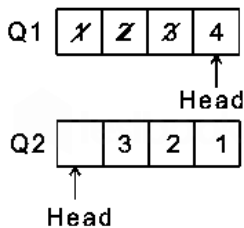
Step 4: Perform the dequeue operation in Q1 at the head position and perform enqueue operation in Q2 at the head position. Performs the same operation for all elements in Q1.

- Dequeue(Q1)
- Enqueue(Q2,2)



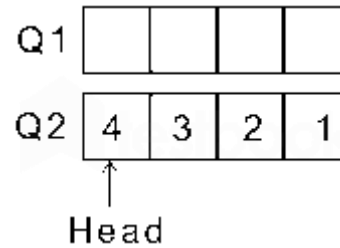
Step 5:

- Dequeue(Q1)
- Enqueue(Q2,3)



Step 6:

- Dequeue(Q1)
- Enqueue(Q2,3)



Thus, there are zero enqueue operations performed on queue Q1.

The minimum number of enqueue operations are 0.

Hence the correct answer is 0.

Q11. Which of the following is not the type of queue?

- (A) Priority queue
- (B) Single-ended queue
- (C) Circular queue
- (D) Ordinary queue

Answer: B

Explanation:

Single Ended Queue is not a type of queue.

Q12. Which of the following data structures can be used to implement queues?

- (A) Stack
- (B) Arrays
- (C) LinkedList
- (D) All of the Above

Answer: D

Explanation:

Stack, Arrays, and LinkedList can be used to implement Queues so all the options are correct.

Q13. Which of the following data structures finds its use in recursion?

- (A) Stack
- (B) Arrays
- (C) LinkedList
- (D) Queues

Answer: A

Explanation:

Stacks find their use in recursion.



Q14. Which of the following data structures allow insertion and deletion from both ends?

- (A) Stack
- (B) Deque
- (C) Queue
- (D) Strings

Answer: B

Explanation:

Deque is a data structure allowing insertion and deletion from both ends.

Q15. Which of the following sorting algorithms provide the best time complexity in the worst-case scenario?

- (A) Merge Sort
- (B) Quick Sort
- (C) Bubble Sort
- (D) Selection Sort

Answer: A

Explanation:

Merge Sort will always have a time complexity of $O(n \cdot \log n)$ which is the best in the worst case among these algorithms.

Q16. What is the maximum number of swaps that can be performed in the Selection Sort algorithm?

- (A) $n - 1$
- (B) n
- (C) 1
- (D) $n - 2$

Answer: A

Explanation:

$n - 1$ swaps are performed at max to sort any array by Selection Sort.

Q17. Which of the following is a Divide and Conquer algorithm?

- (A) Bubble Sort
- (B) Selection Sort
- (C) Heap Sort
- (D) Merge Sort

Answer: D

Explanation:

Merge Sort is a Divide and Conquer algorithm.

Q18. What is the time complexity of the binary search algorithm?

- (A) $O(n)$
- (B) $O(1)$
- (C) $O(\log 2n)$
- (D) $O(n^2)$

Answer: C

Explanation:

Time complexity of binary search algorithm is $O(\log 2n)$.

Q19. What is the best case time complexity of the binary search algorithm?

- (A) $O(1)$
- (B) $O(n)$
- (C) $O(\log 2n)$
- (D) $O(n^2)$

Answer: A

Explanation:

The best-case time complexity occurs when the target element occurs exactly at the middle of the array and the algorithm terminates in 1 move.

Q20. What is the time complexity to insert an element to the front of a LinkedList(head pointer given)?

- (A) $O(n)$
- (B) $O(1)$
- (C) $O(\log n)$
- (D) $O(n \cdot \log n)$

Answer: B

Explanation:

We set the next node to the head of the list, and then return that node as the new head.

Q21. What is the time complexity to insert an element to the rear of a LinkedList(head pointer given)?

- (A) $O(n)$
- (B) $O(1)$
- (C) $O(\log n)$
- (D) $O(n \cdot \log n)$

Answer: A

Explanation:



We need to traverse to the end of the LinkedList and set it next to the new element. So, the traversal will take $O(n)$ time complexity.

Q22. Which of the following can be done with LinkedList?

- (A) Implementation of Stacks and Queues
- (B) Implementation of Binary Trees
- (C) Implementation of Data Structures that can simulate Dynamic Arrays
- (D) All of the above

Answer: D

Explanation:

All the above operations can be performed with LinkedList.

Q23. What is the information, which a LinkedList's Node must store?

- (A) The address of the next node if it exists
- (B) The value of the current node
- (C) Both (A) and (B)
- (D) None of the above

Answer: A

Explanation:

The address of the next node and value of the current node are needed to be stored in a LinkedList's node.

Q24. Worst case time complexity to access an element in a BST can be?

- (A) $O(n)$
- (B) $O(n * \log n)$
- (C) $O(1)$
- (D) $O(\log n)$

Answer: A

Explanation:

In the worst case, we might need to visit all the nodes in the BST.

Q25. Which of the following represents the Postorder Traversal of a Binary Tree?

- (A) Left $\rightarrow$ Right $\rightarrow$ Root
- (B) Left $\rightarrow$ Root $\rightarrow$ Right
- (C) Right $\rightarrow$ Left $\rightarrow$ Root
- (D) Right $\rightarrow$ Root $\rightarrow$ Left

Answer: A

Explanation:

Postorder traversal follows the order of left $\rightarrow$ right $\rightarrow$ root in a binary tree.

Q26. Which of the following statements is true about AVL Trees?

- (A) The difference between the heights of left and right nodes cannot be more than 1.
- (B) The height of an AVL Tree always remains of the order of $O(\log n)$
- (C) AVL Trees are a type of self-balancing Binary Search Trees.
- (D) All of the above.

Answer: D

Explanation:

All the above options are applicable for an AVL Tree.

Q27. In a graph of n nodes and n edges, how many cycles will be present?

- (A) Exactly 1
- (B) At most 1
- (C) At most 2
- (D) Depends on the graph

Answer: A

Explanation:

A tree contains by definition n nodes and $n - 1$ edges, and it is an acyclic graph. When we add 1 edge to the tree, we can form exactly one cycle by adding this edge.

Q28. How is an array initialized in C language?

- (A) `int a[3] = {1, 2, 3};`
- (B) `int a = {1, 2, 3};`
- (C) `int a[] = new int[3]`
- (D) `int a(3) = [1, 2, 3];`

Answer: A

Explanation: An array is initialized in C as per the syntax in Option (A)



Q29. Which of the following is a linear data structure?

- (A) Array
- (B) AVL Trees
- (C) Binary Trees
- (D) Graphs

Answer: A

Explanation:

Array is a linear data structure.

Q30. How can we describe an array in the best possible way?

- (A) The Array shows a hierarchical structure.
- (B) Arrays are immutable.
- (C) Container that stores the elements of similar types
- (D) The Array is not a data structure

Answer: C

Explanation: The answer is c because array stores the elements in a contiguous block of memory of similar types. Therefore, we can say that array is a container that stores the elements of similar types.

Q31. How can we initialize an array in C language?

- (A) `int arr[2]=(10, 20)`
- (B) `int arr(2)={10, 20}`
- (C) `int arr[2] = {10, 20}`
- (D) `int arr(2) = (10, 20)`

Answer: C

Explanation: The answer is c because the values assigned to the array must be enclosed within a curly bracket.

Q32. Which of the following highly uses the concept of an array?

- (A) Binary Search tree
- (B) Caching
- (C) Spatial locality
- (D) Scheduling of Processes

Answer: C

Explanation: The answer is c, i.e., Spatial locality. Here, spatial locality means that the instruction accessed recently, then the nearby memory location would be accessed in the next iteration. As we know that in an array, all the elements are stored in a

contiguous block of memory, so spatial locality is accessed quickly.

Q33. Which of the following is the disadvantage of the array?

- (A) Stack and Queue data structures can be implemented through an array.
- (B) Index of the first element in an array can be negative
- (C) Wastage of memory if the elements inserted in an array are lesser than the allocated size
- (D) Elements can be accessed sequentially.

Answer: C

Explanation: The answer is c. For example, if we have an array of size 10 elements and we have inserted only 5 elements in an array then there is a wastage of 5 memory blocks which cannot be utilized by another variable.

Q34. What is the output of the below code?

```
1. #include <stdio.h>
2. int main()
3. {
4.     int arr[5]={10,20,30,40,50};
5.     printf("%d", arr[5]);
6.
7.     return 0;
8. }
```

- (A) Garbage value
- (B) 10
- (C) 50
- (D) None of the above

Answer: A

Explanation: The answer is a because the indexing in an array starts from 0, so it starts from `arr[0]` to `arr[4]`. If we try to access `arr[5]` then the garbage value will be printed.

Q35. Which one of the following is the size of `int arr[9]` assuming that `int` is of 4 bytes?

- (A) 9
- (B) 36
- (C) 35
- (D) None of the above



Answer: B

Explanation: The answer is b because the size of int type data is 4 bytes. The array stores 9 elements, so the size of the array is $9 \times 4 = 36$ bytes.

Q36. Which one of the following is the process of inserting an element in the stack?

- (A) Insert
- (B) Add
- (C) Push
- (D) None of the above

Answer: C

Explanation: The answer is c. In stack, the process of inserting an element is known as a push operation.

Q37. If the size of the stack is 10 and we try to add the 11th element in the stack then the condition is known as ____

- (A) Underflow
- (B) Garbage collection
- (C) Overflow
- (D) None of the above

Answer: C

Explanation: The answer is c because the stack is full with its 10 elements, and inserting one more element in a stack will lead to the stack overflow.

Q38. Which one of the following is not the application of the stack data structure

- (A) String reversal
- (B) Recursion
- (C) Backtracking
- (D) Asynchronous data transfer

Answer: D

Explanation: The answer is d. The first three options are the stack applications, but option d is not a stack application. The queue data structure is used for synchronization between the processes

Q39. Which of the following is the infix expression?

- (A) $A+B \times C$
- (B) $+A \times BC$
- (C) $ABC+*$
- (D) None of the above

Answer: A

Explanation: The answer is a, i.e., $A+B \times C$ because, in infix notation, all the operators appear between the operands.

Q40. Which of the following is the prefix form of $A+B \times C$?

- (A) $A+(BC*)$
- (B) $+AB \times C$
- (C) $ABC+*$
- (D) $+A \times BC$

Answer: D

Explanation: The answer is d. The prefix notation means all the operators that appear before operand.

To convert the infix expression into a prefix expression, we will move the operator to the left of the parenthesis as shown in the below figure.

Q41. Which of the following is not the correct statement for a stack data structure?

- (A) Arrays can be used to implement the stack
- (B) Stack follows FIFO
- (C) Elements are stored in a sequential manner
- (D) Top of the stack contains the last inserted element

Answer: B

Explanation: The answer is b because Stack does not follow FIFO. It follows LIFO.

Q42. If the elements '1', '2', '3' and '4' are added in a stack, so what would be the order for the removal?

- (A) 1234
- (B) 2134
- (C) 4321
- (D) None of the above

Answer: C

Explanation: The answer is c because stack follows LIFO, which means that the element inserted at the last will be removed first.



Q43. What is the outcome of the prefix expression +, -, *, 3, 2, /, 8, 4, 1?

- (A) 12 (B) 11
(C) 5 (D) 4

Answer: C

Explanation: Reverse of the prefix expression: 1, 4, 8, /, 2, 3, *, -, +

Reading of prefix	Top of the stack	Stack representation
1	1	
4	4	
8	8	
/	(8/4)	
2	2	

3	3	
*	(2*3)	
-	(2*3) - (8/4)	
+	(2*3) - (8/4) + 1	

The infix expression of the above prefix expression is:

$$(2*3) - (8/4) + 1$$

$$6 - 2 + 1 = 5$$

Q44. Which one of the following node is considered the top of the stack if the stack is implemented using the linked list?

- (A) First node
(B) Second node
(C) Last node
(D) None of the above

Answer: A

Explanation: The answer is a, i.e., First node. As we know, that last inserted element in the stack is considered as the top of the stack. Whenever the element is added to the linked list, it is always added at



the beginning of the list. Therefore, we can say that the first node in the linked list is considered as the top of the stack.

Q45. Consider the following stack implemented using stack.

```
1.    #define SIZE 11
2.    struct STACK
3.    {
4.        int arr[SIZE];
5.        int top=-1;
6.    }
```

What would be the maximum value of the top that does not cause the overflow of the stack?

- (A) 8
- (B) 9
- (C) 11
- (D) 10

Answer: D

Explanation: The answer is 10. The maximum size of the array is 11; therefore, we can insert 11 elements in the stack. The top value is initialized by -1, and on every insertion, the top value gets incremented.

Q46. What is another name for the circular queue among the following options?

- (A) Square buffer
- (B) Rectangle buffer
- (C) Ring buffer
- (D) None of the above

Answer: C

Explanation: The circular queue is also known as a ring buffer. In a circular queue, the last element is connected back to the first element of the queue that forms a circle. Therefore, the structure of a circular queue is also known as a ring structure.

Q47. A list of elements in which enqueue operation takes place from one end, and dequeue operation takes place from one end is __

- (A) Binary tree
- (B) Stack
- (C) Queue
- (D) Linked list

Answer: C

Explanation: The answer is Queue. Queue is a data structure in which insertion takes place from one end, and deletion takes place from one end.

Q48. Which one of the following is not the type of the Queue?

- (A) Linear Queue
- (B) Circular Queue
- (C) Double ended Queue
- (D) Single ended Queue

Answer: D

Explanation: The answer is d. i.e., single ended queue. Queue has two ends in which one end is used for the insertion and another end is used for the deletion. Therefore, it is not possible for the Queue to have a single ended queue.

Q49. Which one of the following is the overflow condition if linear queue is implemented using an array with a size MAX_SIZE?

- (A) rear = front
- (B) rear = front+1
- (C) rear=MAX_SIZE -1
- (D) rear = MAX_SIZE

Answer: C

Explanation: The answer is c, i.e., rear=MAX_SIZE-1. As the size of the array is MAX_SIZE, so we can insert the elements till MAX_SIZE-1. If we try to insert the elements of size MAX_SIZE or more than MAX_SIZE in a queue, then it leads to the overflow condition.

Q50. The time complexity of enqueue operation in Queue is __

- (A) O(1)
- (B) O(n)
- (C) O(logn)
- (D) O(nlogn)

Answer: A

Explanation: The answer is a, i.e., O(1). In Queue, the insertion is performed at the rear end, which is directly accessible; therefore, it takes O(1) time to insert an element in a Queue.



Q51. Which of the following that determines the need for the Circular Queue?

- (A) Avoid wastage of memory
- (B) Access the Queue using priority
- (C) Follows the FIFO principle
- (D) None of the above

Answer: A

Explanation: The answer is a, i.e., Avoid wastage of memory. In a linear queue, there are chances of wastage of memory because if the rear is pointing to the last element whereas the front is pointing to the element other than the first element; it means that spaces allocated before the front are free, but it cannot be reused as rear cannot be incremented. In contrast, the last element is connected to the first element in a circular queue; if initial spaces are vacant, then rear can be incremented by using the statement $(\text{rear}+1) \bmod \text{max}$ where max is the size of the array. Therefore, we conclude that the circular queue avoids wastage of memory.

Q52. In the linked list implementation of queue, where will the new element be inserted?

- (A) At the middle position of the linked list
- (B) At the head position of the linked list
- (C) At the tail position of the linked list
- (D) None of the above

Answer: C

Explanation: The answer is c. If the queue is implemented using linked list, then the new element will be inserted at the tail position of the linked list as Queue follows FIFO principle in which new element will always be added at the end of the Queue.

Q53. Which of the following option is true if implementation of Queue is from the linked list?

- (A) In enqueue operation, new nodes are inserted from the beginning and in dequeue operation, nodes are removed from the end.
- (B) In enqueue operation, new nodes are inserted from the end and in dequeue operation, nodes are deleted from the beginning.
- (C) In enqueue operation, new nodes are inserted from the end and in dequeue operation, nodes are deleted from the end.
- (D) Both a and b

Answer: D

Explanation: The answer is d. As we know that Queue has two ends, i.e., one for the insertion and another one for the deletion. If Queue is implemented using Linked list then it can be done in either of the ways.

Q54. The necessary condition to be checked before deletion from the Queue is__

- (A) Overflow
- (B) Underflow
- (C) Rear value
- (D) Front value

Answer: B

Explanation: The answer is b, i.e., Underflow. Before deleting an element from the Queue, we first need to check whether the Queue is empty or not.

Q55. Which of the following principle is used if two elements in the priority queue have the same priority?

- (A) LIFO
- (B) FIFO
- (C) Linear tree
- (D) None of the above

Answer: B

Explanation: The answer is b, i.e., FIFO. In a priority queue, if two or more elements have the same priority then they are arranged based on the FIFO principle.



Q56. Which of the following statement is not true regarding the priority queue?

- (A) Processes with different priority can be easily handled
- (B) Easy to implement
- (C) Deletion is easier
- (D) None of the above

Answer: C

Explanation: The answer is c. i.e., deletion is easier. In worst case, the deletion is not easier as we have to traverse to the n elements until the element to be removed is not found.

Q57. Which of the following data structure allows you to insert the elements from both the ends while deletion from only one end?

- (A) Input-restricted queue
- (B) Output-restricted queue
- (C) Priority queue
- (D) None of the above

Answer: B

Explanation: The answer is b. The output-restricted queue is one of the types of the Deque data structure in which insertion is allowed from both the ends but the deletion is allowed from only one end.

Q58. Which of the following is the time complexity to search an element in the linked list?

- (A) $O(1)$
- (B) $O(n)$
- (C) $O(\log n)$
- (D) $O(n \log n)$

Answer: B

Explanation: The answer is $O(n)$. The worst-case time complexity to search an element in the linked list is $O(n)$ because if we have to find the last element then we need to traverse till the n th node.

Q59. Which of the following statement is not true about the doubly linked list?

- (A) We can traverse in both the directions.
- (B) It requires extra space
- (C) Implementation of doubly linked list is easier than the singly linked list
- (D) It stores the addresses of the next and the previous node

Answer: C

Explanation: The answer is c. The implementation of doubly linked list is complex as compared to singly linked list as it needs to store the addresses of the two nodes, i.e., the previous and the next node. If we insert or delete the node from the doubly linked list then it needs to adjust two pointers, previous and the next pointer.

Q60. Consider the following code

```
1. struct node
2. {
3.     int data;
4.     struct node *next;
5. }
6. node ptr;
```

Which one of the following is the correct option to create a new node?

- (A) `ptr = (node*)malloc(sizeof(node*))`
- (B) `ptr = (node)malloc(sizeof(node))`
- (C) `ptr = (node*)malloc(sizeof(node))`
- (D) None of the above

Answer: C

Explanation: The answer is c, i.e., `ptr = (node*)malloc(sizeof(node))`. In this statement, we have used a `malloc()` function for allocating the memory to the node and `ptr` is a pointer variable that will point to the newly created node.

Q61. What is the maximum number of children that a node can have in a binary tree?

- (A) 3
- (B) 1
- (C) 4
- (D) 2

Answer: D

Explanation: The answer is d. The binary tree can contain utmost two children.

Q62. Which one of the following techniques is not used in the Binary tree?

- (A) Randomized traversal
- (B) Preorder traversal
- (C) Postorder traversal
- (D) Inorder traversal

Answer: A



Explanation: The answer is a. The binary tree contains three traversal techniques, preorder, postorder and inorder traversal.

Q63. Which of the following options is not true about the Binary Search tree?

- (A) The value of the left child should be less than the root node
- (B) The value of the right child should be greater than the root node.
- (C) The left and right sub trees should also be a binary search tree
- (D) None of the above

Answer: D

Explanation: The answer is d. All the three options, i.e., a, b and c are true for the binary search tree. In binary search tree, the left child should be less than the root node and the right child should be greater than the value of the root node.

Q64. How can we define a AVL tree?

- (A) A tree which is binary search tree and height balanced tree.
- (B) A tree which is a binary search tree but unbalanced tree.
- (C) A tree with utmost two children
- (D) A tree with utmost three children

Answer: A

Explanation: The answer is a. An AVL tree is a binary search tree and height balanced tree.

Q65. Why do we prefer Red Black tree over AVL tree?

- (A) Red Black trees are not strictly balanced
- (B) Red black tree requires lesser rotations than AVL tree.
- (C) AVL tree needs more space to store the balance factor.
- (D) Both b and c

Answer: D

Explanation: The answer is d. Red Black tree requires fewer rotations for inserting or deleting the node whereas AVL tree requires multiple rotations to balance the tree. AVL tree stores balance factor per node that

requires space whereas Red Black tree stores 1-bit information.

Q66. What would be the color of newly created node while inserting a new element in a Red black tree?

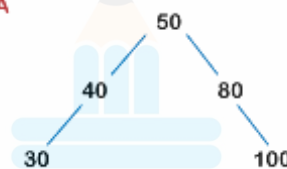
- (A) Black, if the new node is not a root node
- (B) Red, if the new node is not a root node
- (C) Black, if the new node is a root node
- (D) Both b and c

Answer: D

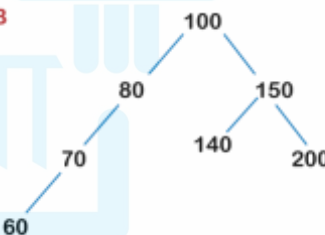
Explanation: The answer is d. The property of Red Black tree is that if the newly created node is a root node then the color of the node would be Black otherwise the color of the node would be Red.

Q67. Identify the AVL tree among the following options?

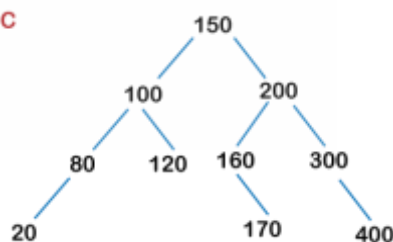
A



B



C



- (A) A
- (B) C
- (C) Both A and C
- (D) B

Answer: C

Explanation: The answer is c. Both A and C are the AVL trees but B is not a AVL tree. As we know that balance factor of every node in AVL tree should be either -1, 0 or 1. But in case of B, the balance factor of node 80 is 2, so it is not a AVL tree.